



UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH FACULTÉ DES SCIENCES ET TECHNIQUES FES

« Ingénierie Electrique et Systèmes Embarqués »

COMPTE RENDU: Traitement Numérique du Signal

Préparé par :

FRANKLIN HAMUNYEMBA
AMANSOUR MOHAMMED

Supervisé par :

Pr. N. ES-SBAI

Introduction générale

Ce rapport présente l'ensemble des travaux pratiques réalisés dans le cadre du module de Traitement Numérique du Signal. L'objectif principal de ces manipulations est de mettre en application, sous MATLAB, les notions fondamentales étudiées en cours, en reliant les modèles théoriques aux résultats obtenus par simulation.

Le premier travail pratique est consacré à trois opérations de base du traitement numérique du signal : la quantification, la convolution discrète et la transformée de Fourier discrète. Il permet d'étudier la représentation numérique des signaux, l'effet du bruit de quantification, la réponse des systèmes linéaires invariants ainsi que l'analyse fréquentielle de différents signaux.

Le deuxième travail pratique porte sur le filtrage numérique. Il s'agit d'analyser et de comparer plusieurs familles de filtres, notamment les filtres à réponse impulsionnelle infinie et les filtres à réponse impulsionnelle finie. Cette partie met en évidence les compromis entre sélectivité, stabilité, phase et complexité de mise en œuvre.

Le troisième travail pratique est dédié aux signaux aléatoires, à leurs propriétés statistiques et à l'estimation spectrale. Il permet d'étudier la moyenne, la variance, les fonctions de corrélation ainsi que différentes méthodes d'estimation de la densité spectrale de puissance, en particulier dans le cas de signaux issus d'un filtrage numérique.

Ainsi, ce rapport a pour but de montrer comment les outils du traitement numérique du signal peuvent être utilisés pour représenter, analyser et interpréter des signaux dans les domaines temporel et fréquentiel. Il met également en évidence l'intérêt des simulations numériques pour valider les résultats théoriques et mieux comprendre le comportement des systèmes étudiés.

Table des matières

I.	TP 1 : QUANTIFICATION, CONVOLUTION & TFD	5
2.1	Introduction	5
2.2	Objectifs du TP	5
2.3	Rappels théoriques	6
2.3.1	Quantification	6
2.3.2	Convolution discrète	6
2.3.3	Transformée de Fourier discrète	7
2.4	Étude de la quantification	7
2.4.1	Signal utilisé	7
2.4.2	Quantification linéaire	8
2.4.3	Bruit de quantification et puissance moyenne	9
2.4.4	Quantification logarithmique	10
2.5	Convolution de signaux discrets	11
2.5.1	Implémentation de la convolution directe	11
2.5.2	Convolution par FFT	12
2.6	Application de la transformée de Fourier discrète	14
2.6.1	Implémentation de la TFD	14
2.6.2	TFD d'une impulsion de Dirac	15
2.6.3	TFD d'une impulsion de Dirac décalée	15
2.6.4	TFD d'une somme de trois sinusoïdes	16
2.6.5	TFD d'un extrait audio	17
2.7	Conclusion du chapitre	18
II.	TP 2: FILTRAGE NUMÉRIQUE	19
2.1	Introduction	19
2.2	Objectifs du TP	19
2.3	Rappels théoriques	20
2.3.1	Généralités sur le filtrage	20
2.3.2	Filtres à réponse impulsionnelle infinie	20
2.3.3	Filtres à réponse impulsionnelle finie	20
2.4	Étude des filtres RII classiques	20
2.4.1	Filtre de Butterworth d'ordre 9	20
2.4.2	Filtre elliptique d'ordre 9	21
2.4.3	Comparaison entre Butterworth et elliptique	23

2.5	Conception d'un filtre RIF passe-bande par la méthode des fenêtres	23
2.5.1	Paramètres de synthèse	23
2.5.2	Programme MATLAB	24
2.5.3	Analyse et interpretation	24
2.6	Comparaison générale entre filtres RII et RIF	25
2.7	Conclusion du chapitre	25
III.	TP 3 : SIGNAUX ALÉATOIRES	26
3.1	Introduction	26
3.2	Objectifs du TP	26
3.3	Rappels théoriques	26
3.3.1	Moyenne et variance	26
3.3.2	Autocorrélation et intercorrélation	27
3.3.3	Densité spectrale de puissance	27
3.3.4	Corrélation des signaux MA et AR	27
3.4	Génération et analyse d'un bruit blanc	28
3.5	Corrélation des signaux MA	30
3.6	Corrélation des signaux AR	32
3.7	Estimation et comparaison de la densité spectrale de puissance	34
3.7.1	DSP théorique	34
3.7.2	Estimation par périodogramme	35
3.7.3	Estimation par la méthode de Welch	36
3.7.4	Estimation par les méthodes paramétriques	37
3.7.5	Comparaison des méthodes	38
3.8	Conclusion du chapitre	38

I. TP 1 : QUANTIFICATION, CONVOLUTION & TFD

2.1 Introduction

Dans ce premier travail pratique, nous avons étudié trois opérations fondamentales du traitement numérique du signal, à savoir la quantification, la convolution discrète et la transformée de Fourier discrète. Ces outils jouent un rôle central dans la représentation, l'analyse et le traitement des signaux numériques.

La quantification permet de représenter un signal continu en amplitude par un nombre fini de niveaux. Elle introduit inévitablement une erreur appelée bruit de quantification, dont l'importance dépend du nombre de bits utilisés. La convolution discrète, quant à elle, permet de déterminer la sortie d'un système linéaire invariant à partir de son entrée et de sa réponse impulsionnelle. Enfin, la transformée de Fourier discrète permet de passer du domaine temporel au domaine fréquentiel afin d'identifier les composantes spectrales d'un signal.

L'objectif de ce chapitre est donc d'implémenter ces différentes opérations sous MATLAB, de visualiser les résultats obtenus, puis d'interpréter les observations à la lumière des rappels théoriques.

2.2 Objectifs du TP

Les objectifs de ce travail pratique sont les suivants :

- Étudier la quantification linéaire d'un signal numérique pour différentes valeurs du nombre de bits ;
- Calculer et représenter le bruit de quantification ainsi que sa puissance moyenne ;
- Comparer la quantification linéaire et la quantification logarithmique ;
- Programmer la convolution discrète de deux signaux ;
- Vérifier la longueur du signal de sortie obtenu par convolution ;
- Calculer la convolution par la méthode fréquentielle en utilisant la FFT ;
- Programmer la TFD d'un signal à partir de sa définition ;
- Appliquer la TFD à différents signaux et interpréter les résultats obtenus.

2.3 Rappels théoriques

2.3.1 Quantification

La quantification consiste à remplacer les amplitudes d'un signal par un nombre fini de niveaux. Pour une quantification uniforme sur $J = 2^B$ niveaux, le pas de quantification est donné par :

$$q = \frac{2}{2^B}$$

dans le cas où le signal est normalisé entre -1 et 1.

La valeur quantifiée est obtenue en remplaçant chaque échantillon par le niveau le plus proche. Le signal quantifié s'écrit alors :

$$x_q(n) = q \left(\left\lfloor \frac{x(n)}{q} \right\rfloor + \frac{1}{2} \right)$$

Le bruit de quantification est défini par :

$$b_q(n) = x(n) - x_q(n)$$

et sa puissance moyenne est donnée par :

$$e = \frac{1}{N} \sum_{n=0}^{N-1} (x(n) - x_q(n))^2$$

Lorsque le nombre de bits augmente, le pas de quantification diminue, ce qui améliore la précision et réduit le bruit de quantification.

2.3.2 Convolution discrète

La convolution discrète de deux signaux $x(n)$ et $g(n)$ est définie par :

$$y(n) = \sum_{m=-\infty}^{+\infty} x(m)g(n-m)$$

Cette opération permet de calculer la sortie d'un système linéaire invariant. Si $x(n)$ est de longueur N et $g(n)$ de longueur M , alors le signal convolué $y(n)$ est de longueur :

$$L = N + M - 1$$

2.3.3 Transformée de Fourier discrète

La transformée de Fourier discrète d'un signal $x(n)$ de longueur N est définie par :

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j\frac{2\pi kn}{N}}$$

La TFD permet de représenter le contenu fréquentiel d'un signal. Son calcul direct nécessite un grand nombre d'opérations, alors que l'algorithme FFT permet une implémentation beaucoup plus rapide.

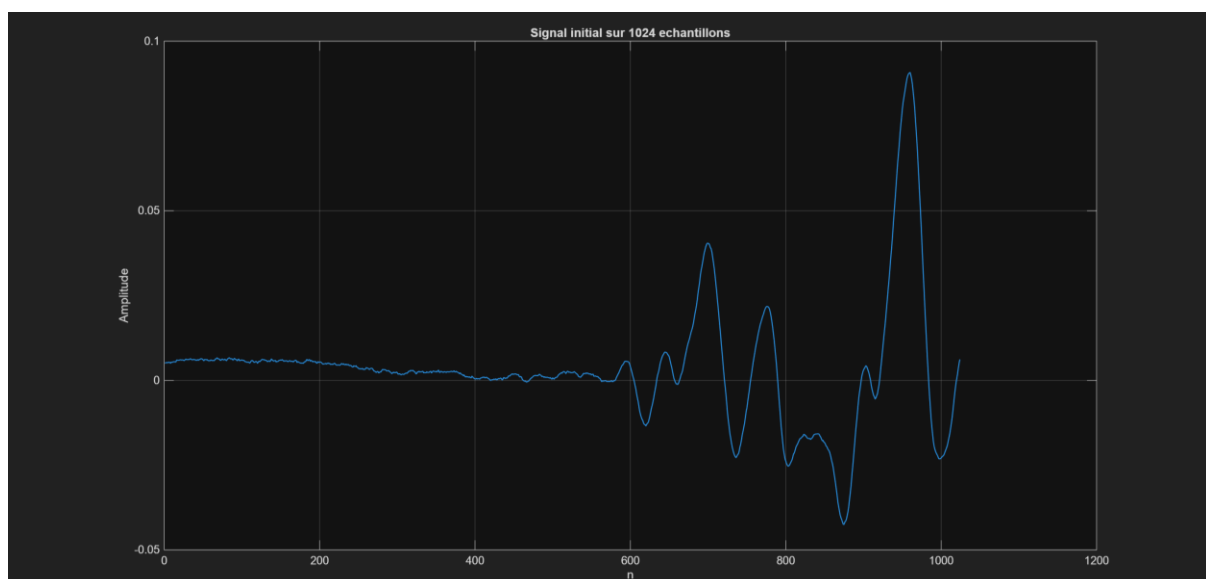
2.4 Étude de la quantification

2.4.1 Signal utilisé

Pour cette partie, nous avons utilisé un extrait sonore de la Mélodie Hongroise de Franz Schubert. Après chargement sous MATLAB, nous avons sélectionné 1024 échantillons afin de travailler sur un segment court mais représentatif du signal.

Le programme utilisé pour charger le signal et afficher le signal initial est le suivant :

```
[s, Fe] = audioread('SCHUBERT.WAV');  
s = s(:,1);  
s = s(1:1024);  
  
figure;  
plot(s);  
grid on;  
title('Signal initial');  
xlabel('n');  
ylabel('Amplitude');
```



L'observation du signal montre qu'il est correctement borné en amplitude et qu'il peut donc être traité par les routines de quantification.

2.4.2 Quantification linéaire

La fonction de quantification linéaire utilisée est la suivante :

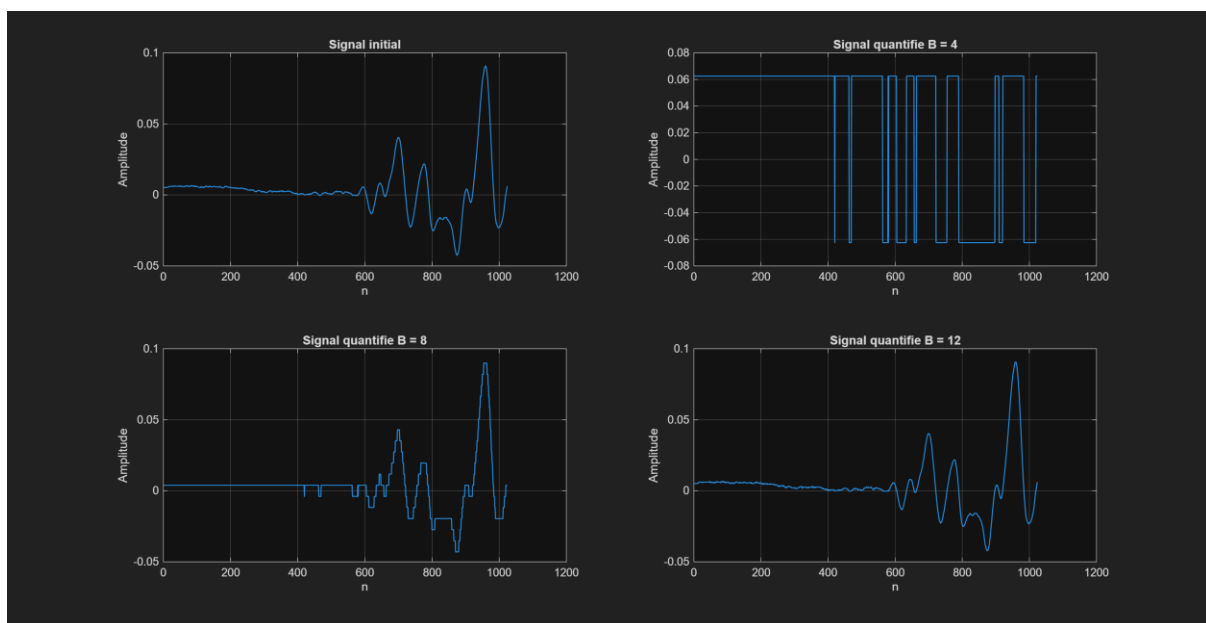
```
function Sres = quantlin(y,B)
    J = 2^B;
    q = 2/J;
    E = floor(y/q);
    Sres = q*(E + 0.5);
end
```

Nous avons appliqué cette quantification pour trois résolutions différentes : $B = 4$, $B = 8$ et $B = 12$.

Le programme correspondant est :

```
signalB4 = quantlin(s,4);
signalB8 = quantlin(s,8);
signalB12 = quantlin(s,12);

figure;
subplot(2,2,1); plot(s); grid on; title('Signal initial');
subplot(2,2,2); plot(signalB4); grid on; title('Signal quantifié B = 4');
subplot(2,2,3); plot(signalB8); grid on; title('Signal quantifié B = 8');
subplot(2,2,4); plot(signalB12); grid on; title('Signal quantifié B = 12');
```



Analyse et interprétation

Les résultats montrent que la quantification à 4 bits produit un signal en marches d'escalier très visibles, ce qui traduit une faible précision. Lorsque le nombre de bits passe à 8, le signal quantifié devient plus proche du signal original. Pour 12 bits, la différence visuelle entre le signal quantifié et le signal initial devient très faible.

Cela s'explique par le fait que l'augmentation du nombre de bits augmente le nombre de niveaux disponibles et réduit ainsi le pas de quantification. Par conséquent, l'erreur introduite sur chaque échantillon devient plus faible.

2.4.3 Bruit de quantification et puissance moyenne

Le bruit de quantification a été calculé par différence entre le signal original et le signal quantifié. La fonction suivante a été utilisée :

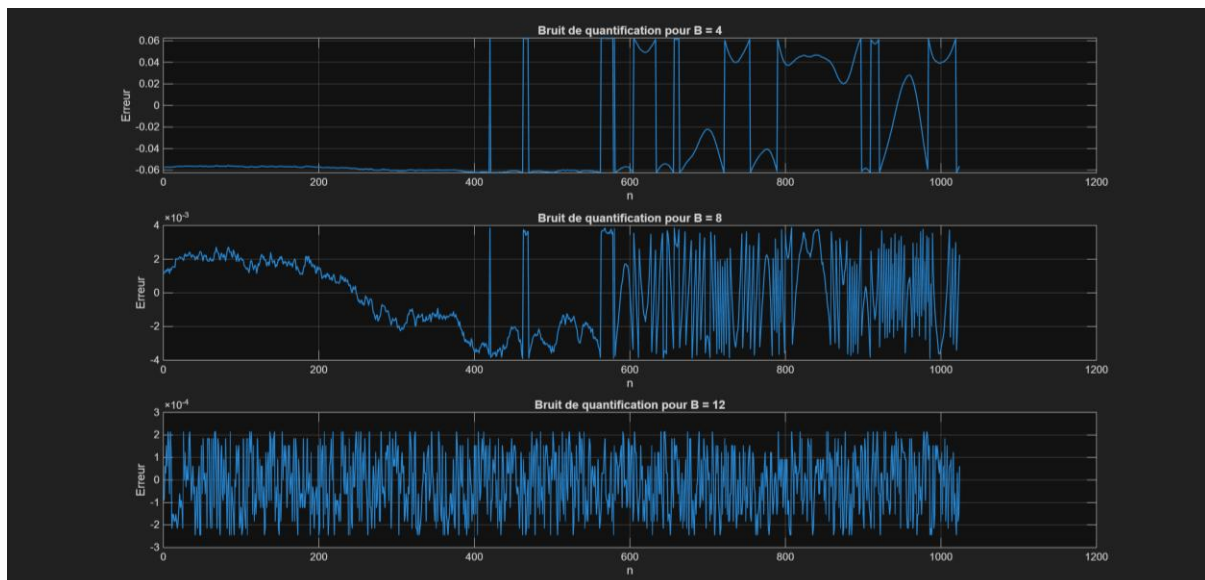
```
function [bruit,e] = testlin(y,B)
    bruit = y - quantlin(y,B);
    e = mean(bruit.^2);
end
```

Le programme de calcul et de visualisation est :

```
[bruitB4,e4] = testlin(s,4);
[bruitB8,e8] = testlin(s,8);
[bruitB12,e12] = testlin(s,12);

disp(['Puissance moyenne pour B = 4 : ', num2str(e4)]);
disp(['Puissance moyenne pour B = 8 : ', num2str(e8)]);
disp(['Puissance moyenne pour B = 12 : ', num2str(e12)]);

figure;
subplot(3,1,1); plot(bruitB4); grid on; title('Bruit de quantification pour B = 4');
subplot(3,1,2); plot(bruitB8); grid on; title('Bruit de quantification pour B = 8');
subplot(3,1,3); plot(bruitB12); grid on; title('Bruit de quantification pour B = 12');
```



Puissance moyenne du bruit de quantification :

B = 4 -> 2.932689e-03

B = 8 -> 4.925886e-06

B = 12 -> 1.983881e-08

Analyse et interprétation

Le bruit de quantification diminue clairement lorsque le nombre de bits augmente. La puissance moyenne de l'erreur est plus élevée pour 4 bits et devient beaucoup plus faible pour 8 puis 12 bits.

Ce résultat confirme directement la relation théorique entre la résolution de quantification et la précision de représentation. Plus B augmente, plus l'écart entre le signal quantifié et le signal d'origine diminue.

2.4.4 Quantification logarithmique

Contrairement à la quantification linéaire, la quantification logarithmique permet de mieux représenter les faibles amplitudes grâce à une loi de compression. Pour cette étude, nous avons utilisé une loi de type μ -law.

La fonction MATLAB utilisée est :

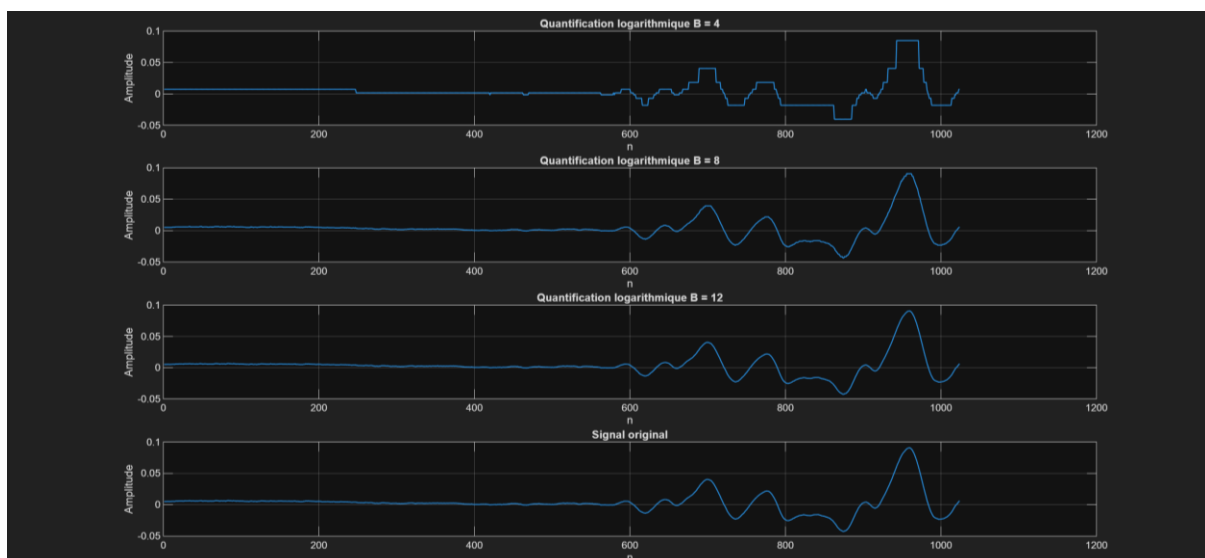
```
function res = quantlog(y,B)
    q = 2/(2^B);
    mu = 255;

    y_log = sign(y).*log(1 + mu*abs(y))/log(1 + mu);
    index = floor(y_log/q);
    res = q*(index + 0.5);
    res = sign(res).*(((1 + mu).^abs(res)) - 1)/mu;
end
```

Le programme de visualisation est le suivant :

```
res1 = quantlog(s,4);
res2 = quantlog(s,8);
res3 = quantlog(s,12);

figure;
subplot(4,1,1); plot(res1); grid on; title('Quantification logarithmique B = 4');
subplot(4,1,2); plot(res2); grid on; title('Quantification logarithmique B = 8');
subplot(4,1,3); plot(res3); grid on; title('Quantification logarithmique B = 12');
subplot(4,1,4); plot(s); grid on; title('Signal original');
```



Analyse et interprétation

Les résultats montrent que la quantification logarithmique fournit une meilleure précision sur les faibles amplitudes que la quantification linéaire. Cela est particulièrement utile pour les signaux audio, dont l'énergie n'est pas uniformément répartie.

Comme pour la quantification uniforme, l'augmentation du nombre de bits améliore également la qualité de restitution. Toutefois, ici la répartition non linéaire des niveaux permet d'obtenir une représentation plus adaptée à la dynamique du signal.

2.5 Convolution de signaux discrets

2.5.1 Implémentation de la convolution directe

On considère les deux signaux :

$$x(n) = 1 \text{ pour } 0 \leq n \leq N - 1$$
$$g(n) = 0.7^n \text{ pour } 0 \leq n \leq M - 1$$

avec $N = 10$ et $M = 2N = 20$.

La fonction MATLAB de convolution discrète utilisée est :

```
function y = convolution(x,g)
    g_flip = fliplr(g);
    y = zeros(1, length(x) + length(g) - 1);

    for n = 1:length(y)
        for m = 1:length(g_flip)
            if (n - m + 1 > 0) && (n - m + 1 <= length(x))
                y(n) = y(n) + x(n - m + 1)*g_flip(m);
            end
        end
    end

    y = fliplr(y);
end
```

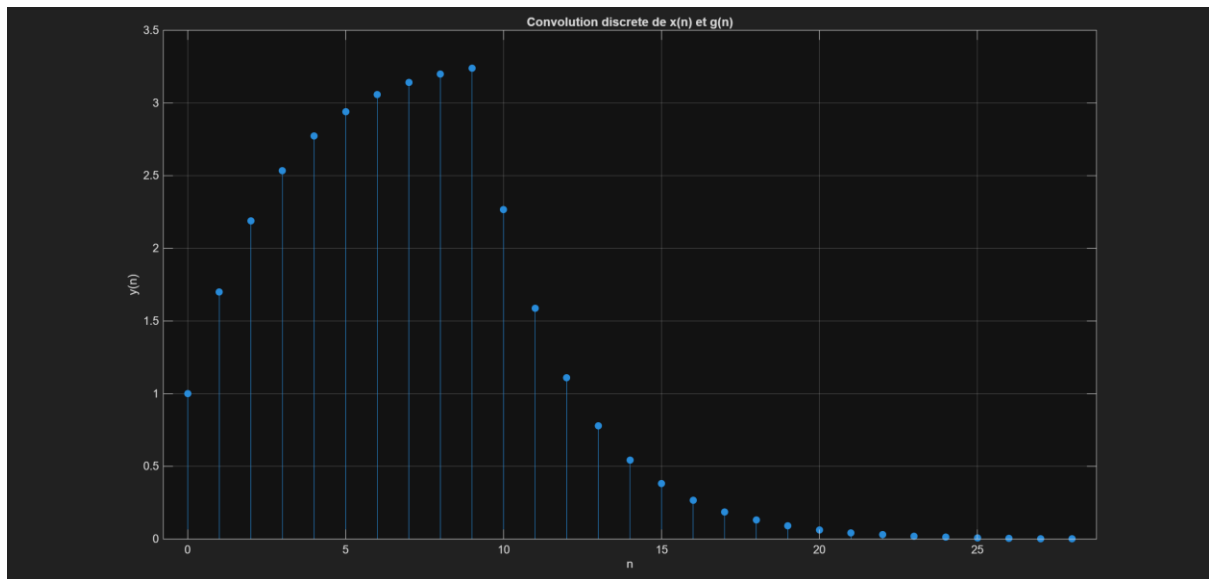
Le programme d'application est :

```
N = 10;
M = 2*N;

x = ones(1,N);
g = 0.7.^(0:M-1);

y = convolution(x,g);

figure;
stem(0:length(y)-1, y, 'filled');
grid on;
title('Convolution discrète de x(n) et g(n)');
xlabel('n');
ylabel('y(n)');
```



Vérification de la longueur

La longueur du signal de sortie est vérifiée par :

```
longueur_y = length(y);
disp(['Longueur de y(n) = ', num2str(longueur_y)]);
disp(['Longueur théorique = ', num2str(N + M - 1)]);
```

```
Longueur de y(n) = 29
Longueur theorique = 29
```

Analyse et interprétation

La convolution obtenue est conforme au comportement attendu d'un système linéaire invariant. La longueur du signal résultant vaut bien $N + M - 1 = 29$, ce qui confirme la validité de l'implémentation.

La forme du signal obtenu traduit le recouvrement progressif entre le signal rectangulaire $x(n)$ et la séquence exponentielle décroissante $g(n)$.

2.5.2 Convolution par FFT

Afin de calculer la convolution plus efficacement, nous avons utilisé la propriété suivante:

$$x(n) * g(n) \Leftrightarrow X(k)G(k)$$

La routine utilisée est :

```
function y = conv_fft(x,g)
    L = length(x) + length(g) - 1;
    X = fft(x,L);
    G = fft(g,L);
    Y = X .* G;
    y = real(ifft(Y));
end
```

Pour $N = 3$ et $M = 6$, les séquences considérées sont :

$$f(n) = e^{0.1n}, 0 \leq n \leq N - 1$$
$$h(n) = e^{-0.1n}, 0 \leq n \leq M - 1$$

Le programme MATLAB est :

```
N = 3; M = 6;

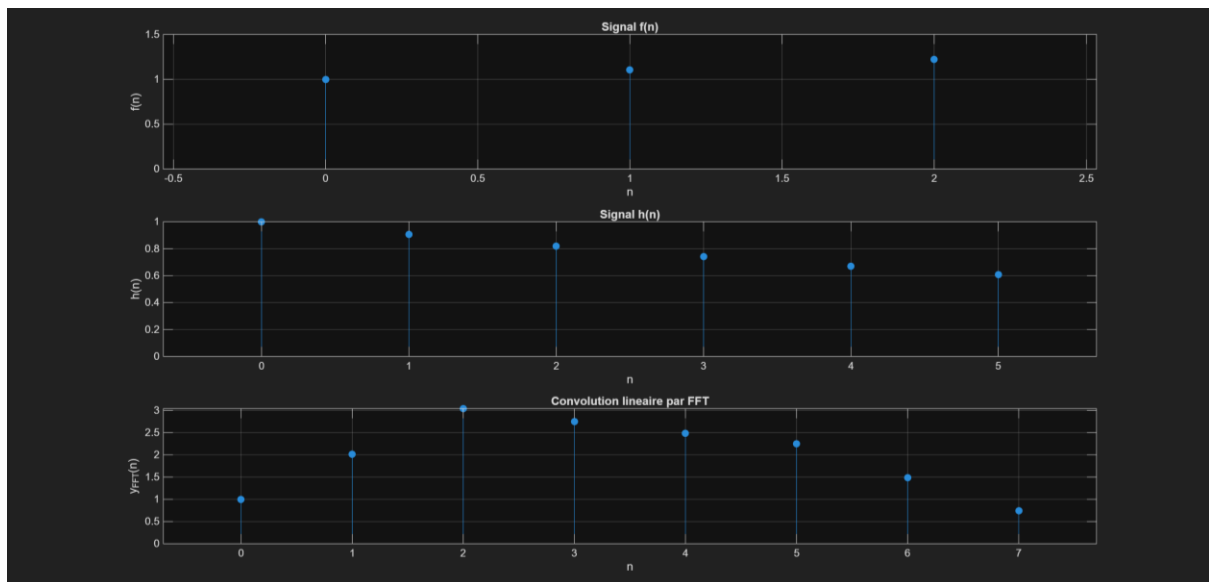
f = exp(0.1*(0:N-1));
h = exp(-0.1*(0:M-1));

y_fft = conv_fft(f,h);
y_conv = conv(f,h);

figure;
subplot(3,1,1);
stem(0:length(f)-1, f, 'filled');
grid on;
title('Signal f(n)');

subplot(3,1,2);
stem(0:length(h)-1, h, 'filled');
grid on;
title('Signal h(n)');

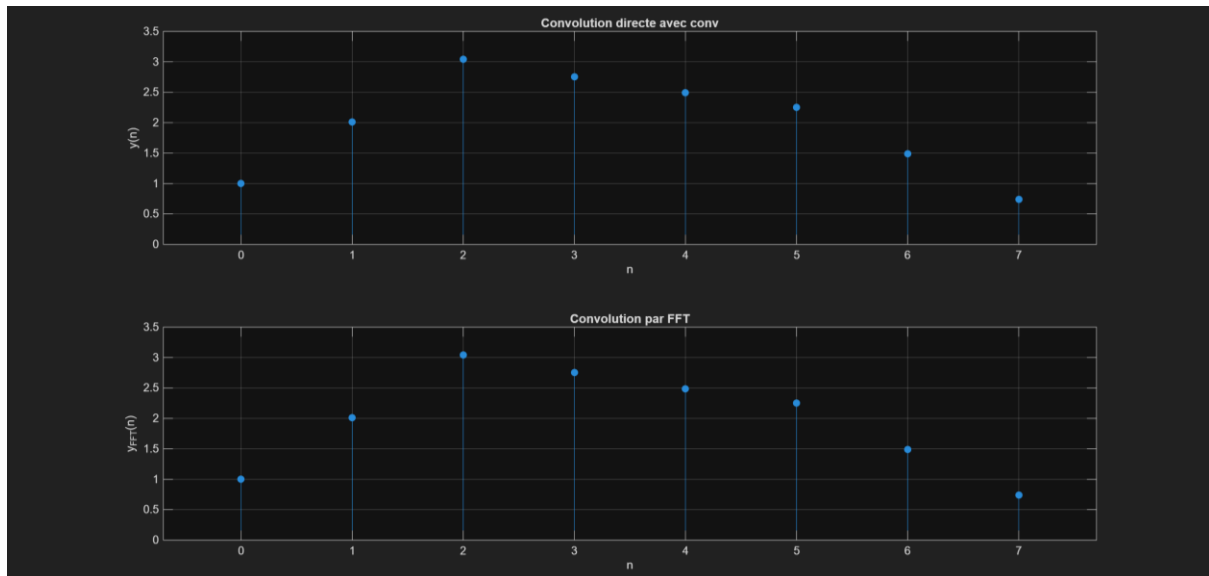
subplot(3,1,3);
stem(0:length(y_fft)-1, y_fft, 'filled');
grid on;
title('Convolution linéaire par FFT');
```



Pour comparer avec la convolution directe :

```
figure;
subplot(2,1,1);
stem(0:length(y_conv)-1, y_conv, 'filled');
grid on;
title('Convolution directe avec conv');

subplot(2,1,2);
stem(0:length(y_fft)-1, y_fft, 'filled');
grid on;
title('Convolution par FFT');
```



Analyse et interprétation

Les deux résultats obtenus sont identiques à l'erreur numérique près. Cela confirme la validité de la méthode fréquentielle. La convolution par FFT est particulièrement avantageuse lorsque les signaux sont longs, car elle permet de réduire considérablement le temps de calcul.

2.6 Application de la transformée de Fourier discrète

2.6.1 Implémentation de la TFD

La fonction MATLAB programmée à partir de la définition théorique est :

```
function X = dft_signal(x)
    N = length(x);
    X = zeros(1,N);

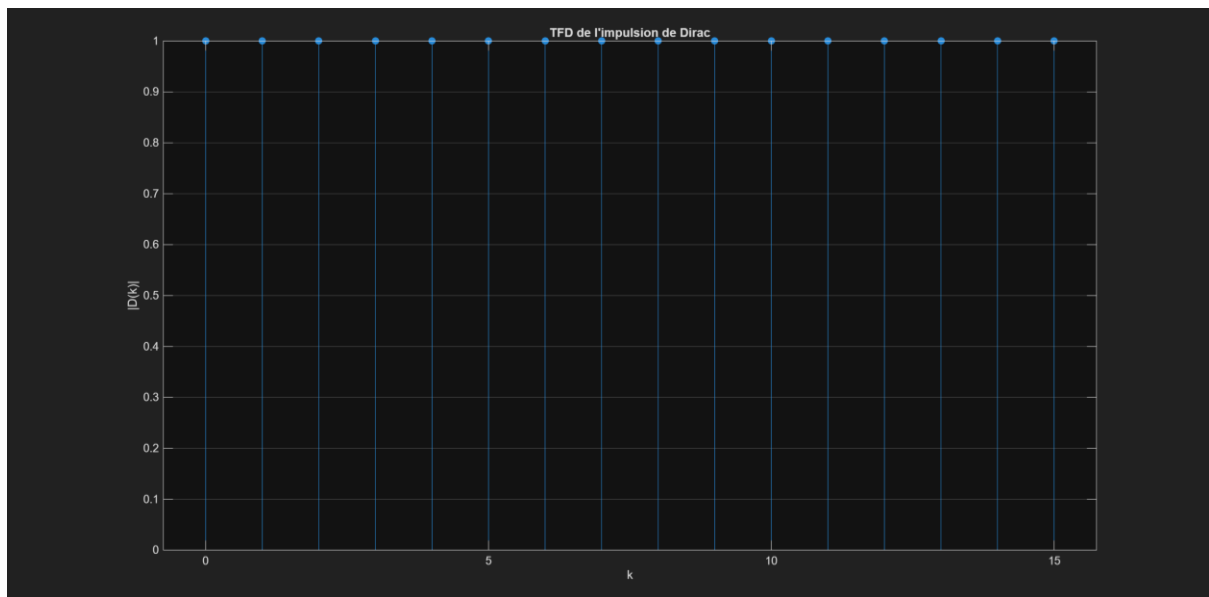
    for k = 1:N
        for n = 1:N
            X(k) = X(k) + x(n)*exp(-1j*2*pi*(k-1)*(n-1)/N);
        end
    end
end
```

2.6.2 TFD d'une impulsion de Dirac

On considère le signal :

```
N = 16;
d = [1 zeros(1,N-1)];
D = dft_signal(d);

figure;
stem(0:N-1, abs(D), 'filled');
grid on;
title('TFD de l'impulsion de Dirac');
xlabel('k');
ylabel('|D(k)|');
```



Analyse et interprétation

Le spectre obtenu est plat. Tous les coefficients ont la même amplitude. Cela confirme qu'une impulsion de Dirac contient toutes les fréquences avec la même contribution.

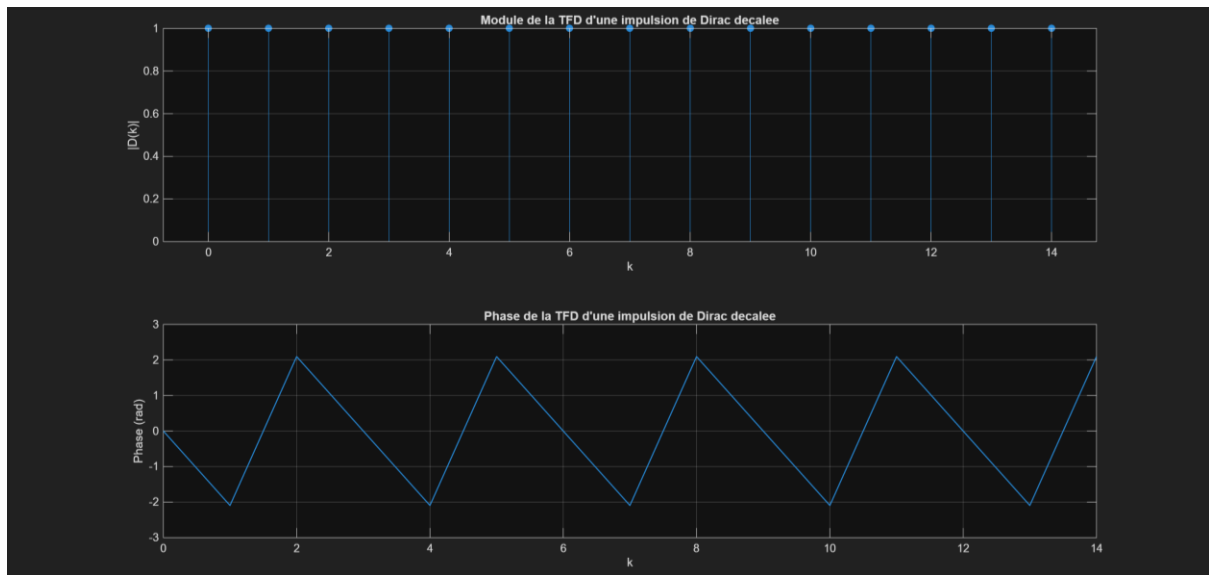
2.6.3 TFD d'une impulsion de Dirac décalée

On considère maintenant :

```
N = 6; M = 10;
d_shifted = [zeros(1,N-1) 1 zeros(1,M-1)];
D_shifted = dft_signal(d_shifted);

figure;
subplot(2,1,1);
stem(0:length(D_shifted)-1, abs(D_shifted), 'filled');
grid on;
title('Module de la TFD d'une impulsion de Dirac décalée');
xlabel('k');
ylabel('|D(k)|');

subplot(2,1,2);
plot(0:length(D_shifted)-1, angle(D_shifted));
grid on;
title('Phase de la TFD d'une impulsion de Dirac décalée');
xlabel('k');
ylabel('Phase');
```



Analyse et interprétation

Le module reste constant, comme pour l'impulsion non décalée, mais la phase varie avec la fréquence. Le décalage temporel se traduit donc par l'apparition d'une phase linéaire dans le domaine fréquentiel.

2.6.4 TFD d'une somme de trois sinusoïdes

Nous avons ensuite appliqué la TFD à une somme de trois sinusoïdes de fréquences 10Hz, 15Hz et 30Hz, d'abord sur $N = 512$ échantillons, puis sur $2N = 1024$ échantillons.

```
fs = 100;
N1 = 512;
t1 = (0:N1-1)/fs;
x1 = sin(2*pi*10*t1) + sin(2*pi*15*t1) + sin(2*pi*30*t1);

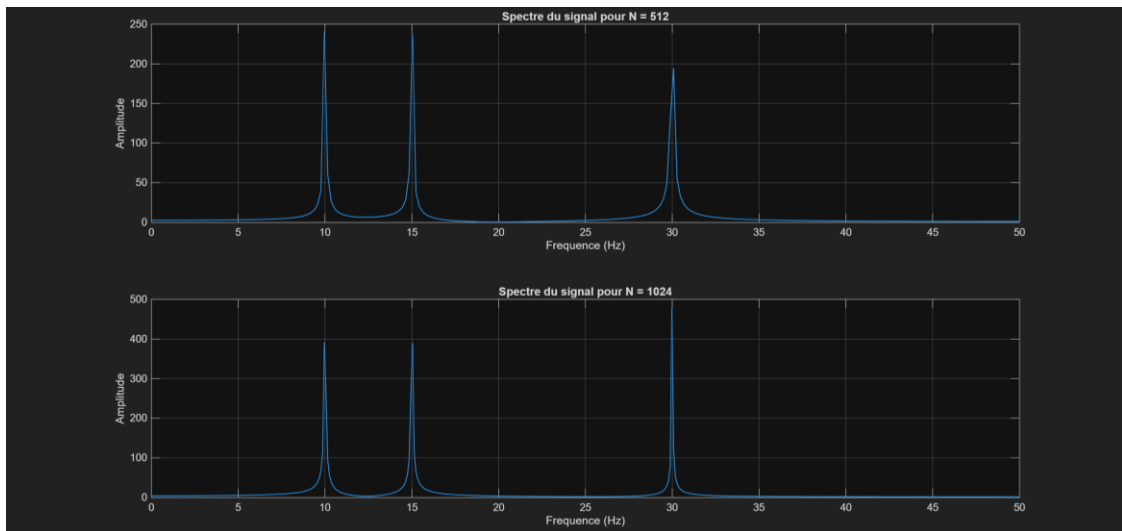
N2 = 1024;
t2 = (0:N2-1)/fs;
x2 = sin(2*pi*10*t2) + sin(2*pi*15*t2) + sin(2*pi*30*t2);

X1 = fft(x1);
X2 = fft(x2);

f1 = (0:N1-1)*fs/N1;
f2 = (0:N2-1)*fs/N2;

figure;
subplot(2,1,1);
plot(f1, abs(X1));
grid on;
title('Spectre du signal pour N = 512');
xlabel('Fréquence (Hz)');
ylabel('Amplitude');

subplot(2,1,2);
plot(f2, abs(X2));
grid on;
title('Spectre du signal pour N = 1024');
xlabel('Fréquence (Hz)');
ylabel('Amplitude');
```



Analyse et interprétation

Les spectres obtenus présentent trois pics principaux situés aux fréquences 10 Hz, 15 Hz et 30 Hz. Ces pics correspondent exactement aux trois composantes du signal.

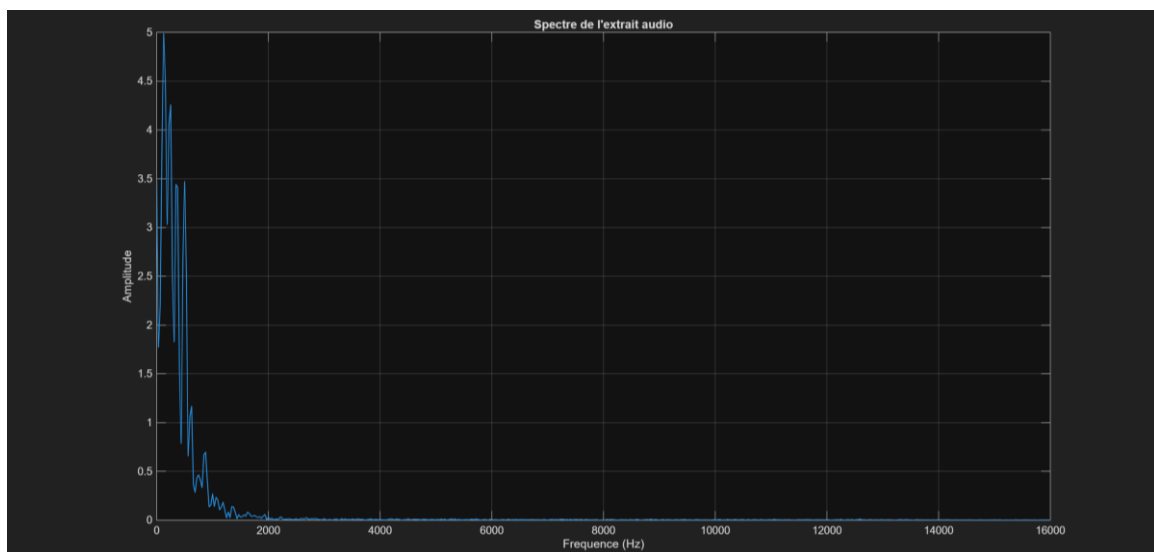
Lorsque le nombre d'échantillons passe de 512 à 1024, la résolution fréquentielle s'améliore. Les pics deviennent plus fins et mieux séparés. Cela montre que l'augmentation de la durée d'observation améliore la précision de l'analyse fréquentielle.

2.6.5 TFD d'un extrait audio

Enfin, nous avons appliqué la TFD à l'extrait audio utilisé dans la partie quantification.

```
X_audio = fft(s);
f_audio = (0:length(s)-1)*Fe/length(s);

figure;
plot(f_audio, abs(X_audio));
grid on;
title('Spectre de l''extrait audio');
xlabel('Fréquence (Hz)');
ylabel('Amplitude');
xlim([0 Fe/2]);
```



Analyse et interprétation

Le spectre obtenu montre que l'énergie du signal audio n'est pas répartie uniformément sur toutes les fréquences. Certaines bandes fréquentielles dominent, ce qui traduit la structure harmonique du signal musical.

Cette observation confirme l'intérêt de la TFD dans l'analyse des signaux réels, en particulier pour l'audio et les applications de traitement fréquentiel.

2.7 Conclusion du chapitre

Ce premier travail pratique nous a permis d'étudier trois notions fondamentales du traitement numérique du signal : la quantification, la convolution discrète et la transformée de Fourier discrète.

L'étude de la quantification a montré que l'augmentation du nombre de bits améliore nettement la précision de représentation et réduit la puissance du bruit de quantification. La comparaison entre quantification linéaire et quantification logarithmique a également mis en évidence l'intérêt d'une loi non linéaire pour les signaux audio.

L'étude de la convolution a permis de vérifier la relation théorique sur la longueur du signal de sortie et de montrer l'équivalence entre la convolution directe et la convolution calculée par FFT.

Enfin, l'application de la TFD à différents signaux a permis de confirmer plusieurs propriétés importantes : spectre plat pour une impulsion de Dirac, influence du décalage temporel sur la phase, identification de composantes sinusoïdales dans un spectre, et amélioration de la résolution fréquentielle lorsque le nombre d'échantillons augmente.

Ainsi, ce chapitre constitue une base essentielle pour la suite des travaux pratiques en traitement numérique du signal.

II. TP 2: FILTRAGE NUMÉRIQUE

2.1 Introduction

Dans ce deuxième travail pratique, nous avons étudié la conception et l'analyse de filtres numériques. Le filtrage constitue une opération essentielle en traitement numérique du signal, puisqu'il permet de sélectionner certaines composantes fréquentielles d'un signal et d'en atténuer d'autres. Il intervient dans de nombreuses applications telles que le débruitage, l'extraction d'information utile, la séparation de bandes fréquentielles ou encore la mise en forme des signaux. Les manipulations demandées dans ce TP portent sur l'étude de filtres à réponse impulsionnelle infinie (RII), en particulier les filtres de Butterworth et elliptiques, ainsi que sur la synthèse d'un filtre à réponse impulsionnelle finie (RIF) par la méthode des fenêtres.

L'objectif est de comparer les comportements fréquentiels de plusieurs structures de filtres, dans les domaines analogique et discret, puis d'analyser l'influence des paramètres de conception sur leurs performances. Une attention particulière est portée à la sélectivité, aux ondulations dans les bandes passante et coupée, à la stabilité et à la phase. Les simulations ont été réalisées sous MATLAB à l'aide des fonctions `butter`, `ellip`, `fir1`, `freqs`, `freqz` et `zplane`, conformément à l'énoncé du TP.

2.2 Objectifs du TP

Les objectifs de ce travail pratique sont les suivants :

- Concevoir et visualiser un filtre de Butterworth d'ordre 9 en continu puis en discret ;
- Concevoir et visualiser un filtre elliptique d'ordre 9 en continu puis en discret ;
- Comparer les réponses fréquentielles des filtres de Butterworth et elliptique ;
- Mettre en évidence les différences entre les filtres RII et RIF ;
- Synthétiser un filtre passe-bande RIF par la méthode des fenêtres ;
- Analyser la réponse impulsionnelle, la réponse fréquentielle et le diagramme pôles-zéros du filtre obtenu ;
- Interpréter les avantages et les limites de chaque type de filtre.

2.3 Rappels théoriques

2.3.1 Généralités sur le filtrage

Un filtre est un système qui modifie le contenu fréquentiel d'un signal. En pratique, les filtres réalisables sont définis à partir de gabarits fréquentiels imposant des contraintes sur les bandes passante et coupée, ainsi que sur les ondulations et la zone de transition. Selon l'application, le filtre peut être passe-bas, passe-haut, passe-bande ou coupe-bande. Dans ce TP, on se limite principalement à l'étude des filtres passe-bas RII et d'un filtre passe-bande RIF.

2.3.2 Filtres à réponse impulsionnelle infinie

Les filtres RII sont caractérisés par une réponse impulsionnelle théoriquement infinie. Ils nécessitent généralement moins de coefficients que les filtres RIF pour atteindre une sélectivité donnée, mais ils présentent souvent une phase non linéaire. Parmi les structures classiques étudiées sous MATLAB, on distingue notamment les filtres de Butterworth et les filtres elliptiques.

Le filtre de Butterworth est connu pour présenter une réponse en amplitude plate dans la bande passante, sans ondulations. Sa transition entre bande passante et bande coupée est relativement progressive. Le filtre elliptique, au contraire, présente des ondulations dans les deux bandes mais offre une transition beaucoup plus rapide. Il est donc plus sélectif que le filtre de Butterworth pour un même ordre.

2.3.3 Filtres à réponse impulsionnelle finie

Les filtres RIF présentent plusieurs avantages importants : ils sont stables, souvent à phase linéaire, et leur implémentation est simple. Leur principal inconvénient réside dans le fait qu'ils nécessitent un ordre plus élevé que les filtres RII pour obtenir des performances comparables en sélectivité. La méthode des fenêtres consiste à tronquer une réponse idéale au moyen d'une fenêtre d'observation finie, comme la fenêtre rectangulaire utilisée dans ce TP.

2.4 Étude des filtres RII classiques

2.4.1 Filtre de Butterworth d'ordre 9

Dans cette première manipulation, nous avons étudié un filtre passe-bas de Butterworth d'ordre 9. La fréquence d'échantillonnage est fixée à $f_e = 1000$ Hz et la fréquence de coupure désirée à $f_c = 300$ Hz. Pour le cas analogique, la pulsation de coupure doit être exprimée en rad/s, soit $\omega_c = 2\pi f_c$. Pour le cas discret, la fréquence doit être normalisée par rapport à la fréquence de Nyquist, d'où $W_n = \frac{f_c}{f_e/2} = 0,6$. L'énoncé demande précisément de visualiser le filtre en continu puis en discret.

Le programme MATLAB utilisé est le suivant :

```

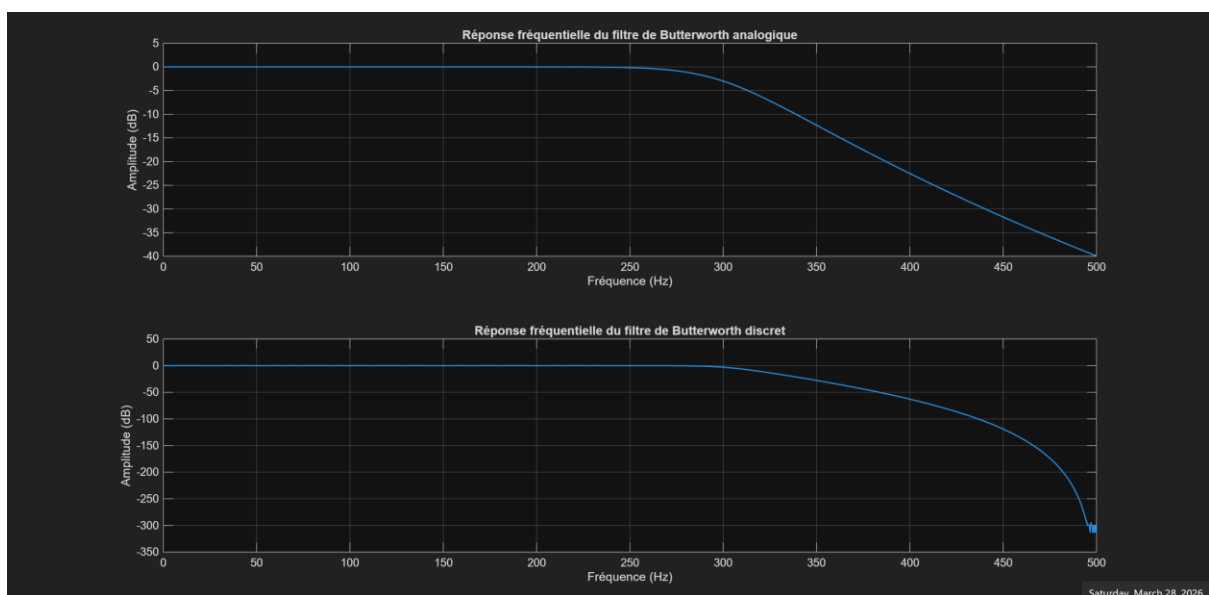
fe = 1000;
fc = 300;
n = 9;

wc = 2*pi*fc;
Wn = fc/(fe/2);

[b,a] = butter(n, wc, 's');
figure;
freqs(b,a);
grid on;
title('Réponse fréquentielle du filtre de Butterworth analogique');

[bn,an] = butter(n, Wn);
figure;
freqz(bn,an);
grid on;
title('Réponse fréquentielle du filtre de Butterworth discret');

```



Analyse et interprétation

La réponse du filtre de Butterworth présente une bande passante régulière, sans ondulations visibles. Ce comportement est conforme à la propriété principale de cette famille de filtres, qui vise une réponse la plus plate possible dans la bande utile. En revanche, la transition entre la bande passante et la bande coupée reste relativement progressive. Cette propriété est visible aussi bien dans le domaine analogique que dans le domaine discret.

Le filtre discret reproduit globalement le comportement du filtre analogique, mais sous une forme adaptée au traitement numérique. La comparaison des deux réponses permet d'observer que la version discrète conserve la forme générale de la réponse tout en s'inscrivant dans le cadre fréquentiel propre aux systèmes échantillonnés.

2.4.2 Filtre elliptique d'ordre 9

Dans la deuxième manipulation, nous avons étudié un filtre elliptique d'ordre 9 avec les spécifications suivantes : $R_p = 3$ dB pour l'ondulation en bande passante et $R_s = 50$ dB

pour l'atténuation en bande coupée. Comme pour le filtre de Butterworth, l'objectif est d'observer la réponse analogique puis la réponse discrète.

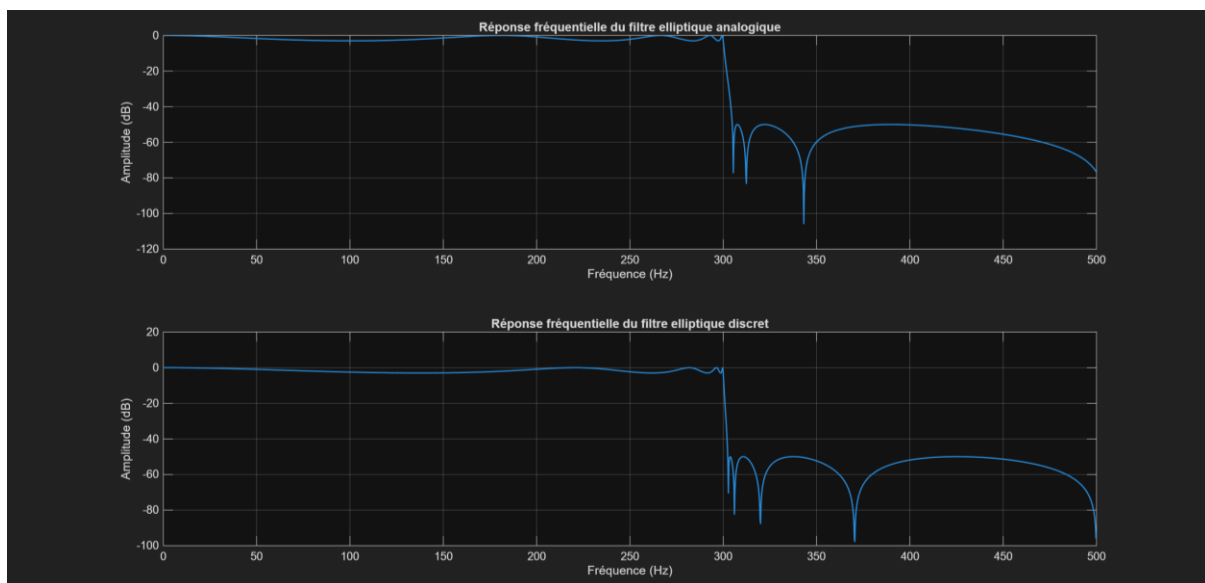
Le programme MATLAB utilisé est le suivant :

```
fe = 1000;
fc = 300;
Rp = 3;
Rs = 50;
n = 9;

wc = 2*pi*fc;
Wn = fc/(fe/2);

[be,ae] = ellip(n, Rp, Rs, wc, 's');
figure;
freqs(be,ae);
grid on;
title('Réponse fréquentielle du filtre elliptique analogique');

[ben,aen] = ellip(n, Rp, Rs, Wn);
figure;
freqz(ben,aen);
grid on;
title('Réponse fréquentielle du filtre elliptique discret');
```



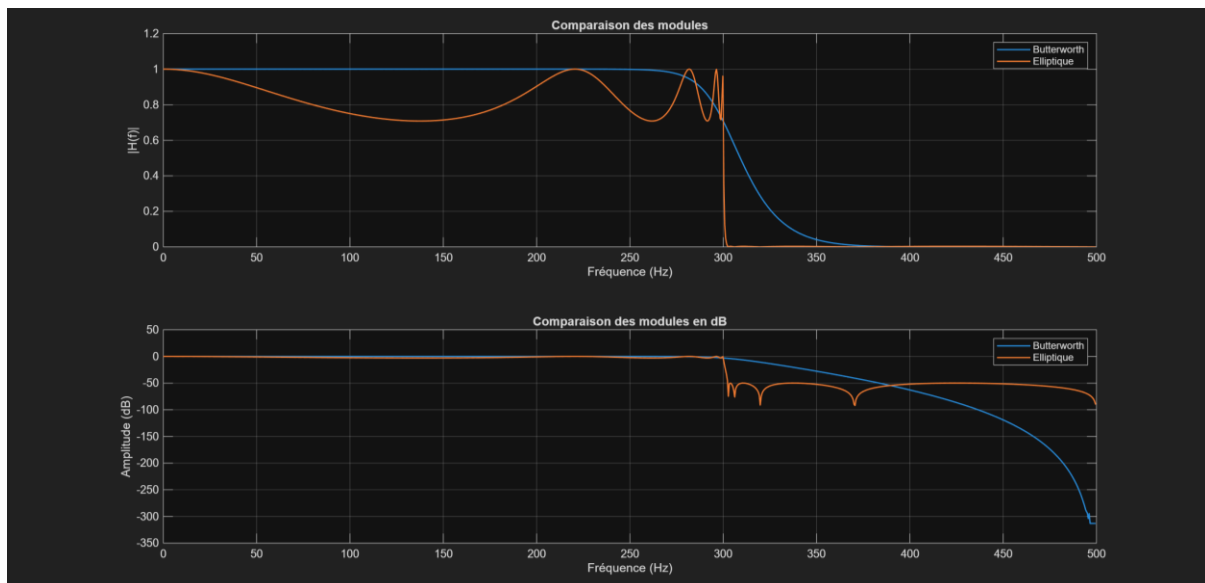
Analyse et interprétation

Les courbes obtenues montrent que le filtre elliptique possède une transition nettement plus rapide que celle du filtre de Butterworth. Il est donc plus sélectif pour un ordre identique. Cependant, cette amélioration de la sélectivité s'accompagne d'ondulations visibles dans la bande passante et dans la bande coupée. Ce comportement correspond exactement aux caractéristiques théoriques des filtres elliptiques.

Dans le domaine discret, on observe que la structure générale est conservée. Le filtre elliptique apparaît ainsi comme un bon choix lorsque l'on cherche une transition étroite entre bande passante et bande coupée, au prix d'une réponse moins régulière que celle du filtre de Butterworth.

2.4.3 Comparaison entre Butterworth et elliptique

La comparaison des deux familles de filtres permet de dégager les observations suivantes. Le filtre de Butterworth offre une réponse plus lisse dans la bande passante, mais sa décroissance est plus lente. Le filtre elliptique, quant à lui, présente une meilleure sélectivité mais introduit des ondulations. Le choix entre les deux dépend donc du compromis recherché entre régularité de la réponse et rapidité de transition. Ces différences sont explicitement mises en avant dans le compte rendu initial et dans le document de TP.



Dans le cadre de cette manipulation, le filtre elliptique est plus performant si l'objectif est d'obtenir une séparation rapide entre les bandes. En revanche, si l'on privilégie une réponse plus uniforme dans la bande passante, le filtre de Butterworth reste plus approprié.

2.5 Conception d'un filtre RIF passe-bande par la méthode des fenêtres

2.5.1 Paramètres de synthèse

La seconde partie du TP consiste à synthétiser un filtre passe-bande RIF à l'aide de la méthode des fenêtres. Les spécifications imposées sont les suivantes :

$$f_{c1} = 500 \text{ Hz}, f_{c2} = 1000 \text{ Hz}, f_e = 4000 \text{ Hz}, NbCoeff = 47$$

Les fréquences de coupure normalisées sont donc :

$$f_{c1N} = \frac{f_{c1}}{f_e/2}, f_{c2N} = \frac{f_{c2}}{f_e/2}$$

Le filtre est obtenu par la fonction `fir1` avec une fenêtre rectangulaire, comme demandé dans le sujet.

2.5.2 Programme MATLAB

Le programme MATLAB utilisé est le suivant :

```
fc1 = 500;
fc2 = 1000;
fe = 4000;

fc1N = fc1/(fe/2);
fc2N = fc2/(fe/2);

NbCoeff = 47;
N = NbCoeff - 1;

h = fir1(N, [fc1N fc2N], 'band', rectwin(NbCoeff), 'scale');

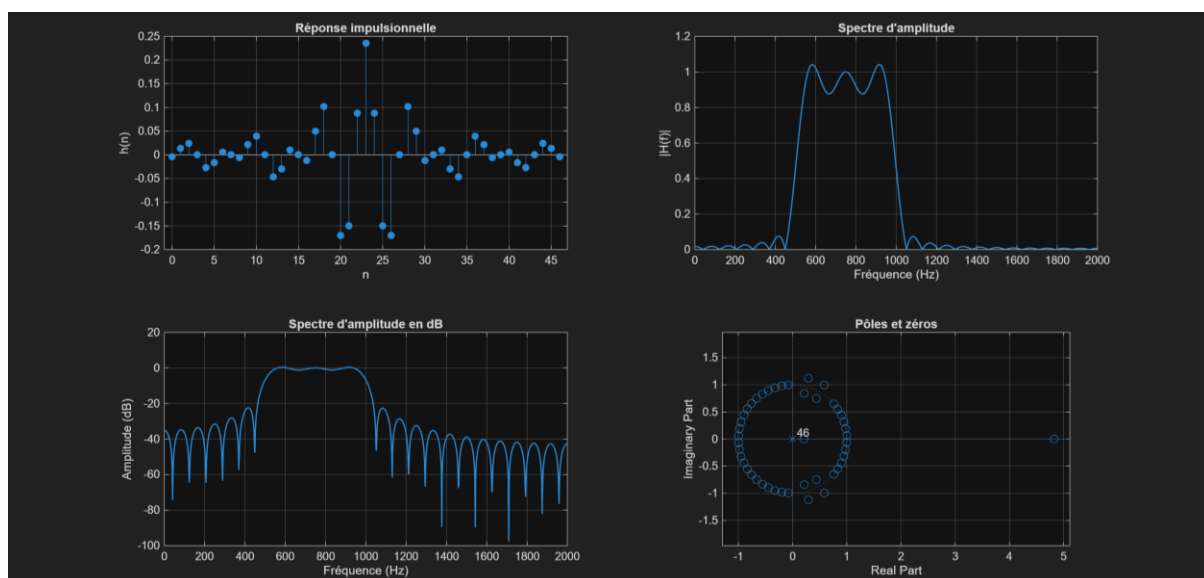
[H,F] = freqz(h, 1, 512, fe);

figure;
subplot(2,2,1);
stem(h, 'b');
grid on;
title('Réponse impulsionnelle du filtre RIF');

subplot(2,2,2);
plot(F, abs(H));
grid on;
title('Spectre d''amplitude');

subplot(2,2,3);
plot(F, 20*log10(abs(H)));
grid on;
title('Spectre d''amplitude en dB');

subplot(2,2,4);
zplane(h,1);
grid on;
title('Pôles et zéros du filtre RIF');
```



2.5.3 Analyse et interpretation

La réponse impulsionnelle obtenue est symétrique, ce qui traduit une phase linéaire. Cette propriété constitue l'un des principaux avantages des filtres RIF. Le spectre

d'amplitude montre clairement une bande passante comprise entre les fréquences désirées, avec une atténuation en dehors de cette zone. Le tracé en décibels permet de mieux apprécier le niveau d'atténuation dans les bandes d'arrêt. Ces observations figurent également dans le compte rendu initial.

Le diagramme pôles-zéros confirme par ailleurs la stabilité du filtre. Les zéros sont répartis de manière cohérente avec la nature passe-bande du système, tandis que l'absence de pôles autres que ceux liés à la structure RIF confirme le caractère non récursif du filtre. Ce résultat illustre bien la différence fondamentale entre filtres RII et RIF.

2.6 Comparaison générale entre filtres RII et RIF

Les filtres RII étudiés dans ce chapitre présentent l'avantage d'obtenir une bonne sélectivité avec un ordre relativement faible. Ils sont donc efficaces lorsqu'on souhaite réduire le nombre de coefficients de calcul. En revanche, ils introduisent généralement une phase non linéaire et exigent davantage d'attention sur les propriétés structurelles du filtre. Les filtres de Butterworth et elliptiques en sont des exemples classiques, avec des compromis différents entre régularité et sélectivité.

Le filtre RIF conçu par fenêtre rectangulaire est plus simple à interpréter, stable par construction, et présente une phase linéaire. Cependant, pour obtenir des performances fréquentielles précises, il nécessite souvent un nombre plus élevé de coefficients. Ce compromis entre simplicité, stabilité, phase et sélectivité doit être pris en compte dans le choix du filtre selon l'application visée.

2.7 Conclusion du chapitre

Ce deuxième travail pratique nous a permis d'étudier plusieurs méthodes de conception de filtres numériques et d'en comparer les principales propriétés. L'analyse des filtres de Butterworth et elliptiques a montré que les filtres RII peuvent offrir des réponses fréquentielles performantes avec un ordre modéré, mais qu'ils diffèrent par leur comportement dans les bandes passante et coupée. Le filtre de Butterworth se distingue par l'absence d'ondulations dans la bande passante, tandis que le filtre elliptique offre une transition plus rapide au prix d'ondulations plus marquées.

La synthèse du filtre RIF passe-bande par la méthode des fenêtres a permis de mettre en évidence les avantages de cette famille de filtres, notamment la stabilité et la phase linéaire. L'étude de la réponse impulsionnelle, du spectre fréquentiel et du diagramme pôles-zéros a confirmé la cohérence des résultats obtenus sous MATLAB. Ce chapitre a ainsi permis de mieux comprendre les différences fondamentales entre filtres RII et RIF, ainsi que les compromis pratiques liés à leur utilisation en traitement numérique du signal.

III. TP 3 : SIGNAUX ALÉATOIRES

3.1 Introduction

Dans ce troisième travail pratique, nous avons étudié les propriétés statistiques des signaux aléatoires ainsi que plusieurs méthodes d'analyse et d'estimation spectrale. Contrairement aux signaux déterministes, un signal aléatoire ne peut pas être décrit exactement à chaque instant ; il est caractérisé à travers des grandeurs statistiques telles que la moyenne, la variance, les fonctions de corrélation et la densité spectrale de puissance (DSP).

L'objectif principal de ce TP est d'analyser un bruit blanc gaussien, puis d'étudier l'effet d'un filtrage numérique sur ses propriétés temporelles et fréquentielles. Nous avons également comparé différentes méthodes d'estimation de la DSP afin d'évaluer leur comportement en termes de régularité, de résolution et de fidélité par rapport à la réponse théorique.

Ce travail permet ainsi de relier les outils statistiques du domaine temporel aux outils spectraux du domaine fréquentiel, ce qui constitue une étape essentielle en traitement numérique du signal.

3.2 Objectifs du TP

Les objectifs de ce travail pratique sont les suivants :

- Générer un bruit blanc gaussien centré de variance donnée ;
- Estimer la moyenne et la variance d'une séquence aléatoire ;
- Calculer et interpréter les estimateurs biaisé et non biaisé de l'autocorrélation ;
- Étudier la corrélation des signaux MA et AR ;
- Mettre en évidence l'influence de la longueur d'observation sur les estimateurs ;
- Calculer la densité spectrale de puissance théorique d'un signal filtré ;
- Comparer les différentes méthodes d'estimation spectrale.

3.3 Rappels théoriques

3.3.1 Moyenne et variance

Pour un signal discret $x(n)$, la moyenne et la variance sont définies respectivement par :

$$m_x = E[x(n)]$$
$$\sigma_x^2 = E[(x(n) - m_x)^2]$$

Dans la pratique, lorsque l'on dispose d'un nombre fini d'échantillons, on utilise des estimateurs calculés à partir de la séquence observée.

3.3.2 Autocorrélation et intercorrélation

La fonction d'autocorrélation d'un signal $x(n)$ est donnée par :

$$R_{xx}(k) = E[x(n)x(n+k)]$$

Elle mesure le degré de dépendance entre les échantillons d'un signal séparés par un retard k .

De même, la fonction d'intercorrélation entre deux signaux $x(n)$ et $y(n)$ s'écrit :

$$R_{xy}(k) = E[x(n)y(n+k)]$$

Pour une séquence finie, on utilise en général deux estimateurs classiques de l'autocorrélation : l'estimateur biaisé et l'estimateur non biaisé. Le premier est plus stable, tandis que le second respecte mieux la définition théorique pour les grands retards.

3.3.3 Densité spectrale de puissance

La densité spectrale de puissance d'un signal représente la répartition de sa puissance en fonction de la fréquence. Elle est reliée à l'autocorrélation par la transformée de Fourier :

$$S_{xx}(f) = \mathcal{F}\{R_{xx}(k)\}$$

Dans ce TP, plusieurs méthodes d'estimation de la DSP sont étudiées :

- Le périodogramme, qui est simple mais très fluctuant ;
- La méthode de Welch, qui effectue une moyenne sur plusieurs segments ;
- Les méthodes paramétriques de Yule-Walker et de Burg, fondées sur un modèle autorégressif.
-

3.3.4 Corrélation des signaux MA et AR

Un signal MA (Moving Average) est obtenu par passage d'un bruit blanc dans un filtre à réponse impulsionnelle finie. Dans le cas étudié ici, on considère :

$$H_1(z) = 0.5 + 0.5z^{-1}$$

Si $x(n)$ est un bruit blanc centré de variance σ_x^2 , alors $y(n)$ est la moyenne glissante de deux échantillons consécutifs. On s'attend donc à une intercorrélation $R_{xy}(k)$ non nulle seulement pour les retards voisins, et à une autocorrélation $R_{yy}(k)$ concentrée autour de $k = -1, 0, 1$.

Un signal AR (Auto-Regressive) est obtenu par passage d'un bruit blanc dans un filtre récursif. Dans ce TP, on considère :

$$H_2(z) = \frac{1}{1 - az^{-1}}, a = 0.75$$

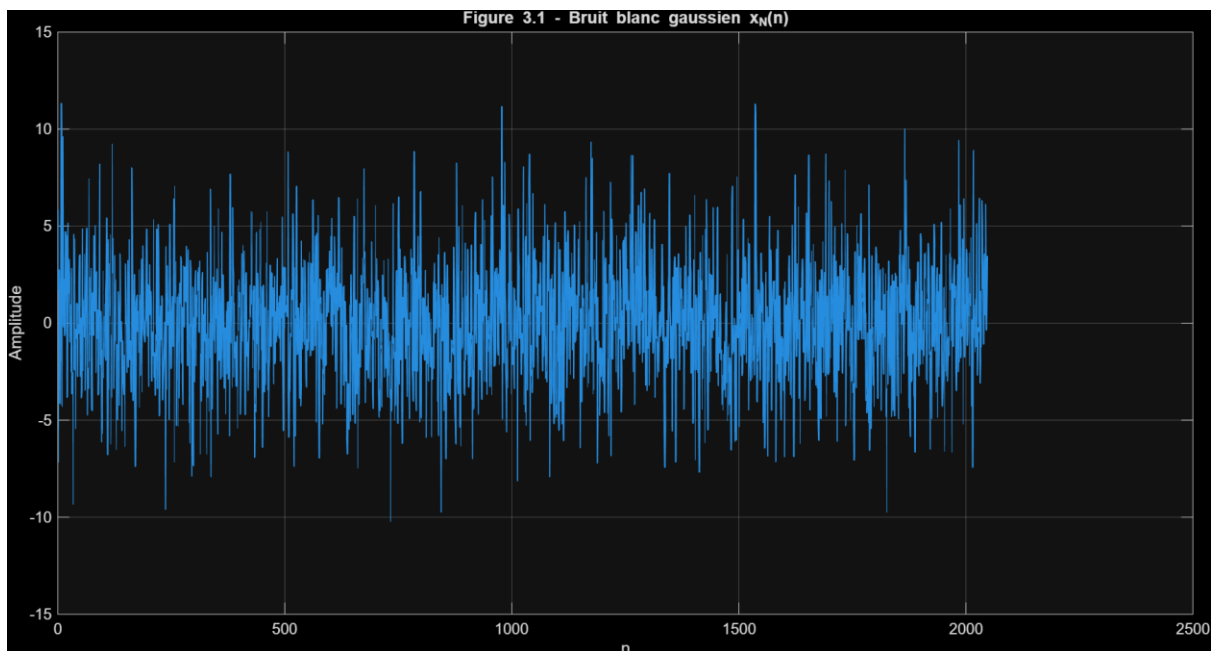
Dans ce cas, la fonction d'autocorrélation théorique décroît exponentiellement avec le retard. Cela permet d'observer clairement l'effet de la mémoire du système sur la structure du signal de sortie.

3.4 Génération et analyse d'un bruit blanc

Dans cette première partie, nous avons généré un bruit blanc gaussien centré de variance $\nu = 10$, comportant $N = 2048$ échantillons. Le programme MATLAB utilisé est le suivant :

```
N = 2048;
v = 10;
xN = normrnd(0, sqrt(v), 1, N);

figure;
plot(0:N-1, xN);
grid on;
title('Bruit blanc gaussien x_N(n)');
xlabel('n');
ylabel('Amplitude');
```



L'allure temporelle du signal obtenu est irrégulière et ne présente aucune structure apparente. Les échantillons fluctuent autour de zéro avec une dispersion compatible avec la variance imposée. Cette observation est cohérente avec le comportement attendu d'un bruit blanc gaussien.

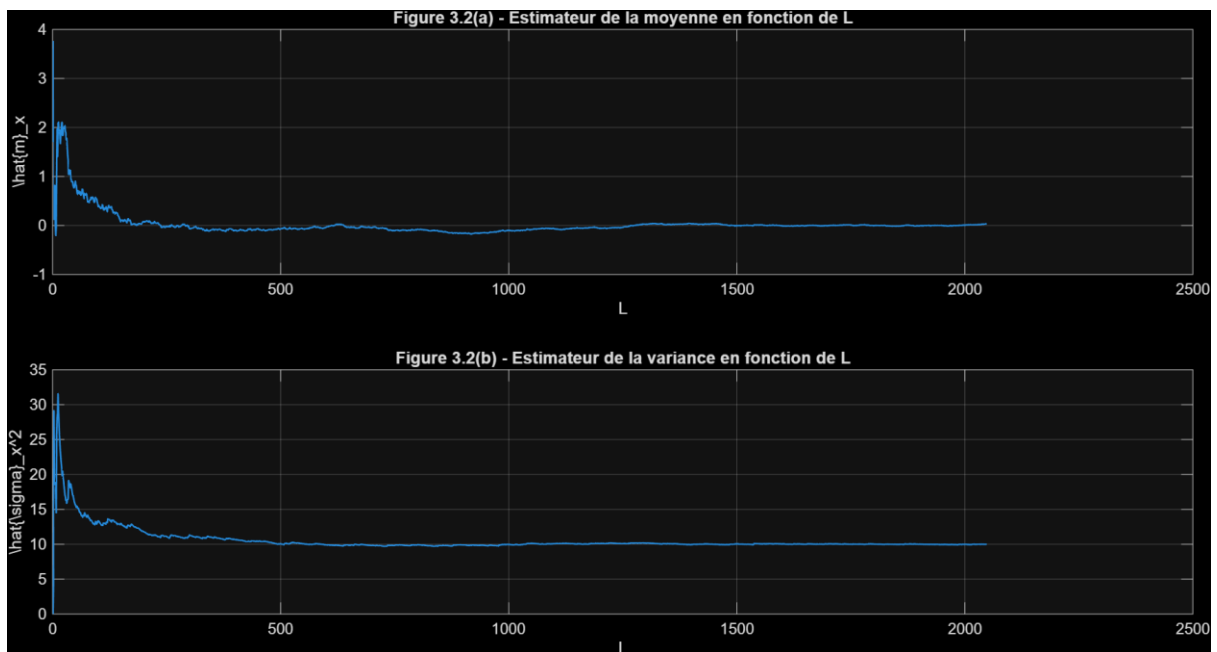
Afin de mieux vérifier les propriétés statistiques du signal, nous avons ensuite calculé les estimateurs de la moyenne et de la variance de la séquence $x_L(n)$ formée par les L premiers échantillons de $x_N(n)$, pour L variant de 1 à N . Le programme MATLAB correspondant est :

```
m_est = zeros(1,N);
v_est = zeros(1,N);

for L = 1:N      xL = xN(1:L);
    m_est(L) = mean(xL);
    v_est(L) = var(xL,1);
end

figure;
subplot(2,1,1);
plot(1:N, m_est);
grid on;
title('Estimateur de la moyenne en fonction de L');
xlabel('L');
ylabel('\hat{m}_x');

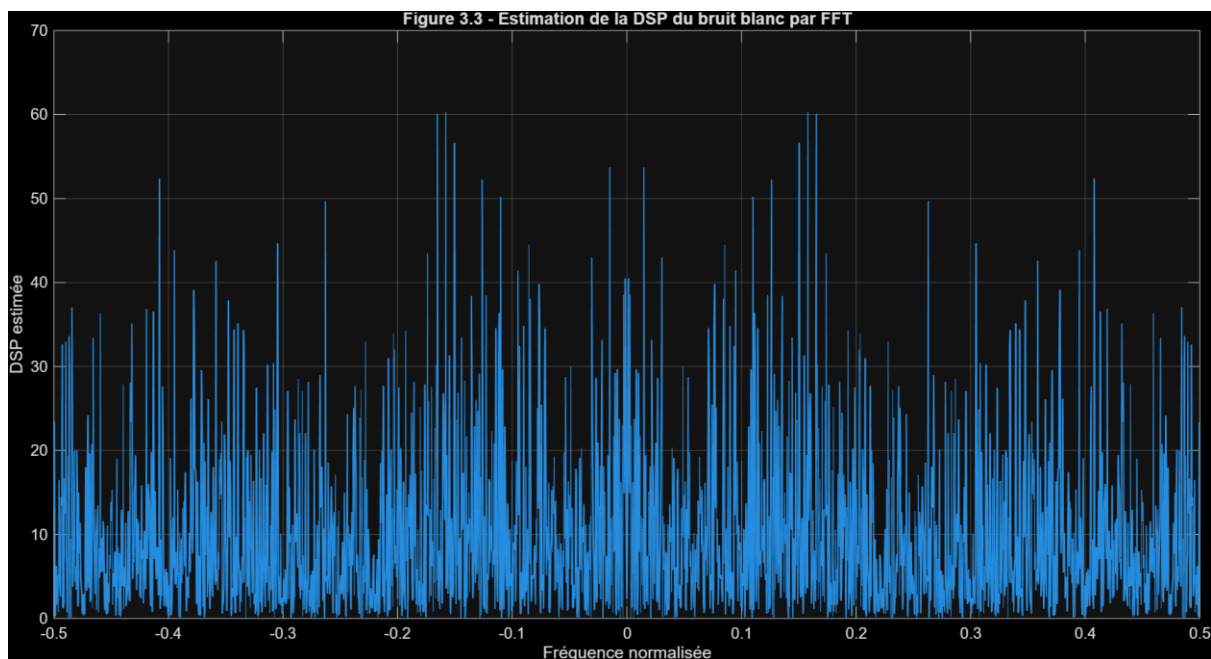
subplot(2,1,2);
plot(1:N, v_est);
grid on;
title('Estimateur de la variance en fonction de L');
xlabel('L');
ylabel('\hat{\sigma}_x^2');
```



Les courbes obtenues montrent que, lorsque L augmente, l'estimateur de la moyenne tend vers zéro, tandis que l'estimateur de la variance se stabilise autour de la valeur théorique imposée. Cela confirme le caractère stationnaire du bruit blanc étudié et montre que la qualité des estimateurs s'améliore avec la taille de l'échantillon.

Nous avons enfin estimé la densité spectrale de puissance du bruit blanc à partir de la transformée de Fourier :

```
X = fft(xN);  
Sx = (abs(X).^2)/N;  
f = (0:N-1)/N;  
  
figure;  
plot(f, fftshift(Sx));  
grid on;  
title('Estimation de la DSP du bruit blanc par FFT');  
xlabel('Fréquence normalisée');  
ylabel('DSP estimée');
```



La DSP d'un bruit blanc étant théoriquement uniforme, on s'attend à une répartition relativement plate de la puissance sur l'ensemble du domaine fréquentiel. En pratique, la courbe obtenue présente des fluctuations aléatoires dues à la longueur finie du signal, mais reste globalement compatible avec le comportement attendu.

3.5 Corrélation des signaux MA

Dans cette partie, nous avons étudié la sortie d'un filtre MA de fonction de transfert :

$$H_1(z) = 0.5 + 0.5z^{-1}$$

Le signal de sortie est obtenu par filtrage du bruit blanc $x_N(n)$:

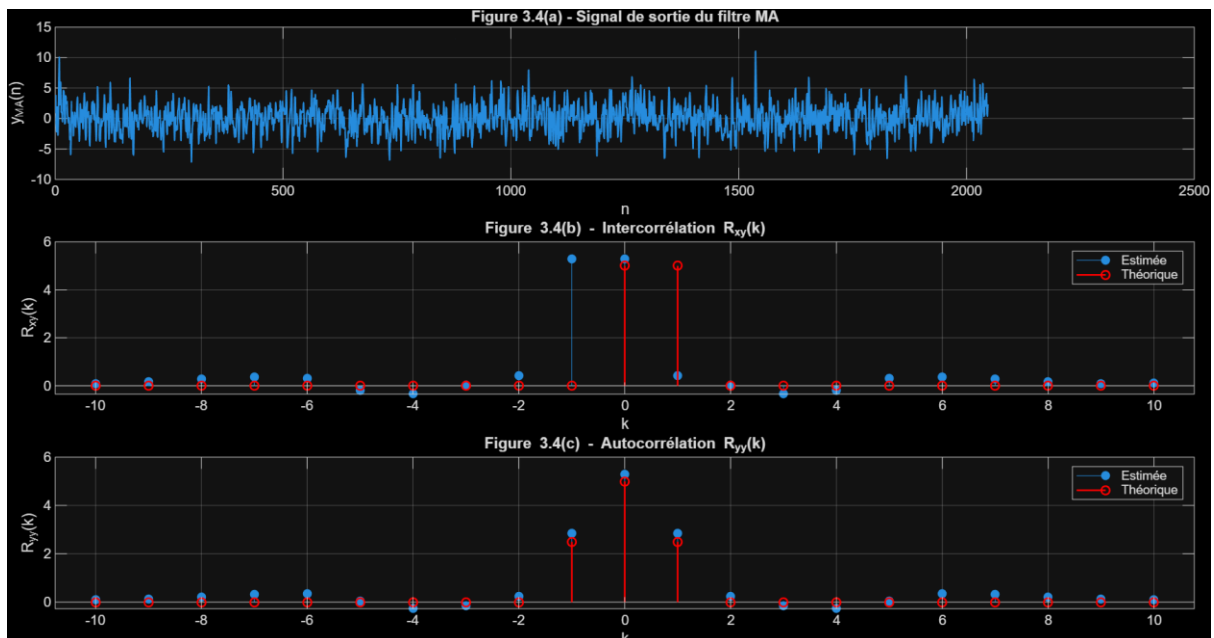
```
b1 = [0.5 0.5];
a1 = 1;
yMA = filter(b1, a1, xN);

m_y = mean(yMA);
[Rxy, lags_xy] = xcorr(xN, yMA, 'unbiased');
[Ryy, lags_yy] = xcorr(yMA, 'unbiased');

figure;
subplot(3,1,1);
plot(0:N-1, yMA);
grid on;
title('Signal de sortie du filtre MA');
xlabel('n');
ylabel('y_{MA}(n)');

subplot(3,1,2);
stem(lags_xy, Rxy, 'filled');
grid on;
title('Estimateur non biaisé de l''intercorrrelation R_{xy}(k)');
xlabel('k');
ylabel('R_{xy}(k)');

subplot(3,1,3);
stem(lags_yy, Ryy, 'filled');
grid on;
title('Estimateur non biaisé de l''autocorrrelation R_{yy}(k)');
xlabel('k');
ylabel('R_{yy}(k)');
```



Théoriquement, pour un bruit blanc centré de variance σ_x^2 , on obtient une intercorrrelation non nulle uniquement pour $k = 0$ et $k = 1$, et une autocorrélation de $y(n)$ non nulle essentiellement pour $k = -1$, $k = 0$ et $k = 1$. Les estimations obtenues sous MATLAB présentent bien cette structure : l'énergie de la corrélation est concentrée

autour de l'origine, ce qui traduit le fait que le filtre MA n'introduit qu'une mémoire très courte.

Cette partie met donc en évidence l'influence directe de la réponse impulsionnelle du filtre sur la forme des fonctions de corrélation.

3.6 Corrélation des signaux AR

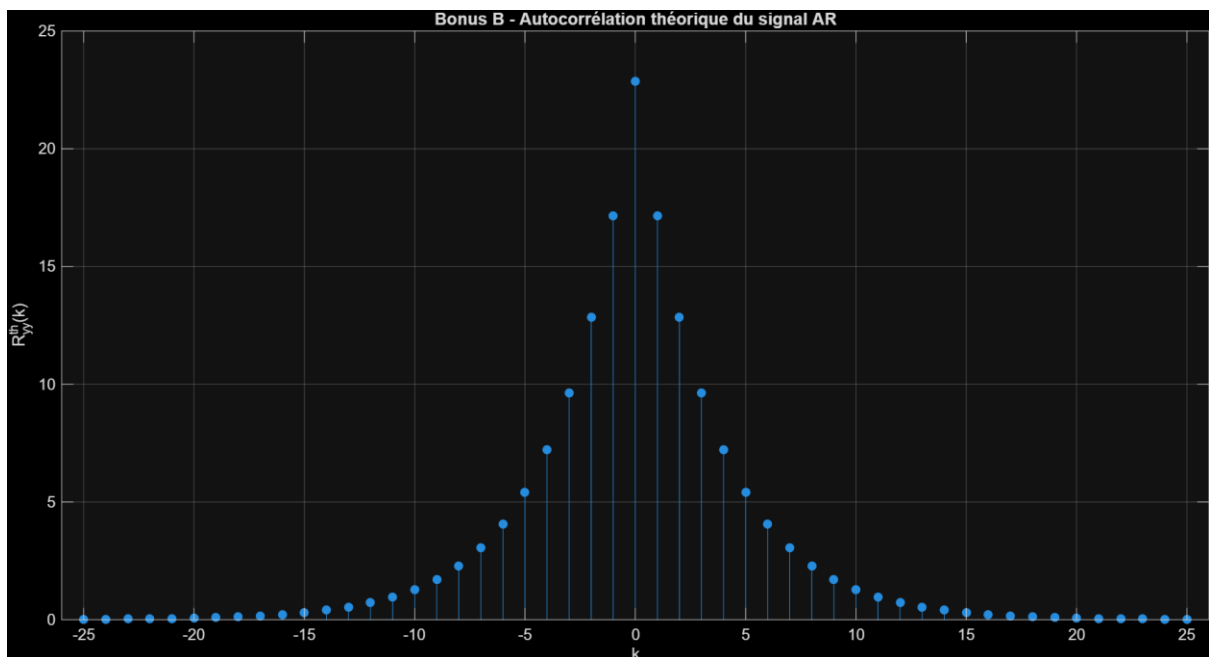
Nous avons ensuite étudié un signal AR obtenu par passage du bruit blanc dans le filtre récursif :

$$H_2(z) = \frac{1}{1 - 0.75z^{-1}}$$

Le signal de sortie est calculé par :

```
a = 0.75;  
b2 = 1;  
a2 = [1 -a];  
yAR = filter(b2, a2, xN);
```

La fonction d'autocorrélation théorique de ce signal décroît exponentiellement avec le retard. Avant d'étudier les estimateurs statistiques obtenus à partir d'une séquence finie, nous avons d'abord tracé cette autocorrélation théorique afin de disposer d'une référence de comparaison.



La courbe obtenue met en évidence une décroissance progressive avec le retard, ce qui traduit la mémoire introduite par le filtre récursif. Cette référence théorique permet ensuite d'évaluer la qualité des estimateurs biaisé et non biaisé pour différentes longueurs d'observation.

Afin d'étudier l'influence de la longueur d'observation sur les estimateurs, nous avons considéré les séquences $y_L(n)$ correspondant aux L premiers échantillons de $y_{AR}(n)$, avec :

$$L = 25, 100, 200, 2000$$

Le programme MATLAB utilisé est le suivant :

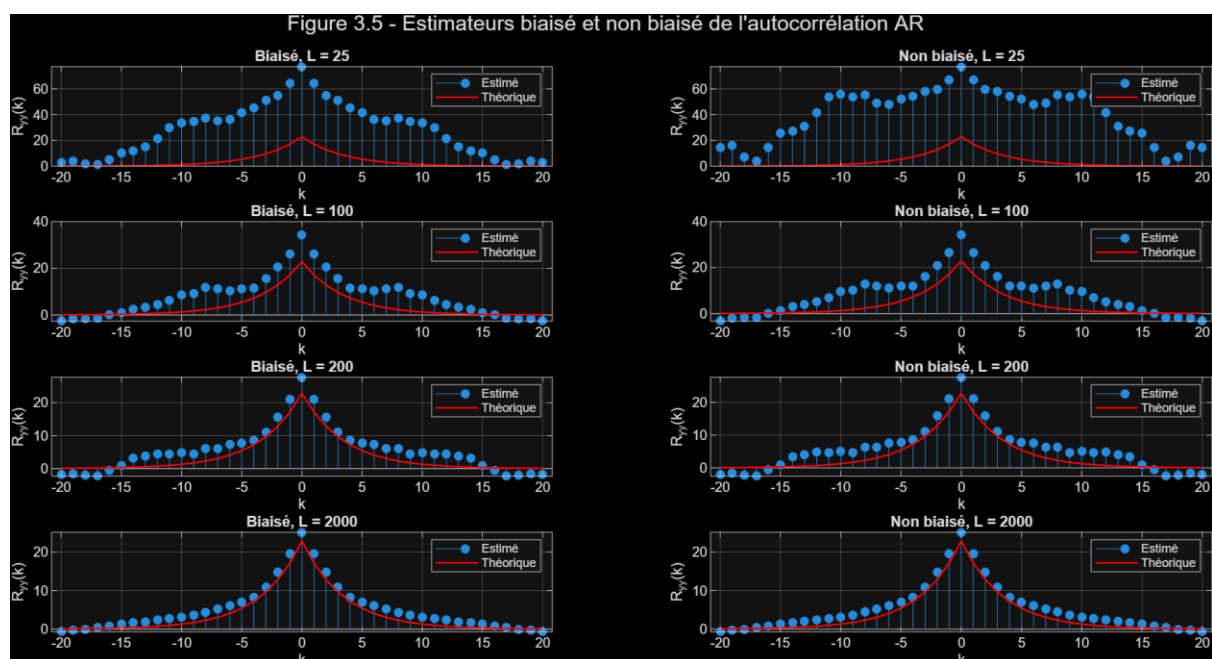
```
L_values = [25 100 200 2000];
k = -50:50;
Rth = a.^abs(k);

figure;
for i = 1:length(L_values)
    L = L_values(i);
    yL = yAR(1:L);

    [Rb, lagsb] = xcorr(yL, 'biased');
    [Runb, lagsu] = xcorr(yL, 'unbiased');

    subplot(length(L_values),2,2*i-1);
    stem(lagsb, Rb, 'filled');
    hold on;
    plot(k, Rth, 'r', 'LineWidth', 1.2);
    grid on;
    title(['Autocorrélation biaisée, L = ', num2str(L)]);
    xlabel('k');
    ylabel('R_{yy}(k)');

    subplot(length(L_values),2,2*i);
    stem(lagsu, Runb, 'filled');
    hold on;
    plot(k, Rth, 'r', 'LineWidth', 1.2);
    grid on;
    title(['Autocorrélation non biaisée, L = ', num2str(L)]);
    xlabel('k');
    ylabel('R_{yy}(k)');
end
```



Les résultats montrent que, pour de faibles longueurs d'observation, les estimateurs sont fortement fluctuants et s'éloignent de la courbe théorique. Lorsque L augmente, les courbes estimées deviennent plus régulières et se rapprochent nettement du comportement théorique exponentiel. L'estimateur biaisé apparaît en général plus stable, tandis que l'estimateur non biaisé présente des fluctuations plus importantes pour les grands retards.

Cette expérience met clairement en évidence l'effet de la taille d'échantillon sur la qualité des estimations statistiques.

3.7 Estimation et comparaison de la densité spectrale de puissance

Dans cette dernière partie, nous avons étudié la densité spectrale de puissance d'un signal filtré $y_N(n)$ obtenu à partir d'un bruit blanc centré, en utilisant le filtre :

$$H(z) = \frac{1}{1 - 2.2137z^{-1} + 2.9403z^{-2} - 2.1697z^{-3} + 0.9606z^{-4}}$$

Le signal est généré par :

```
b = 1;
a = [1 -2.2137 2.9403 -2.1697 0.9606];
yN = filter(b, a, xN);
```

3.7.1 DSP théorique

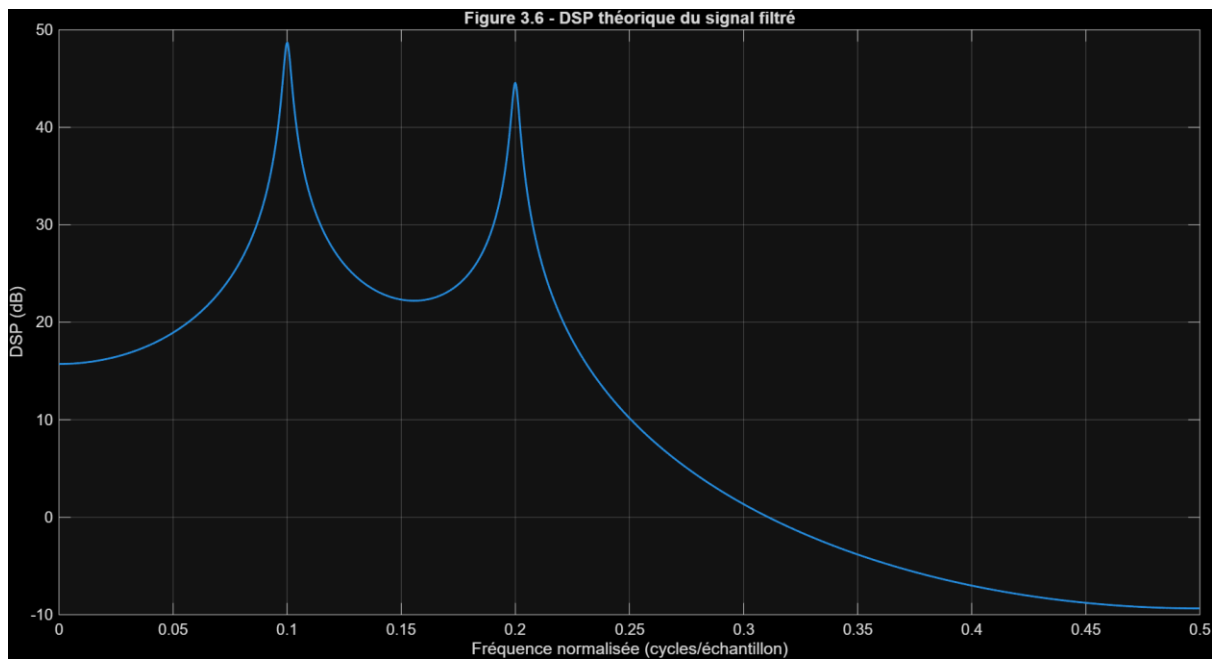
La DSP théorique de la sortie s'écrit :

$$S_{yy}(f) = \sigma_x^2 |H(e^{j2\pi f})|^2$$

Elle a été calculée avec la fonction freqz :

```
[H, w] = freqz(b, a, 1024);
S_theo = v * abs(H).^2;

figure;
plot(w/pi, 10*log10(S_theo), 'LineWidth', 1.5);
grid on;
title('DSP théorique du signal filtré');
xlabel('Fréquence normalisée (\times \pi rad/échantillon)');
ylabel('DSP (dB)');
```

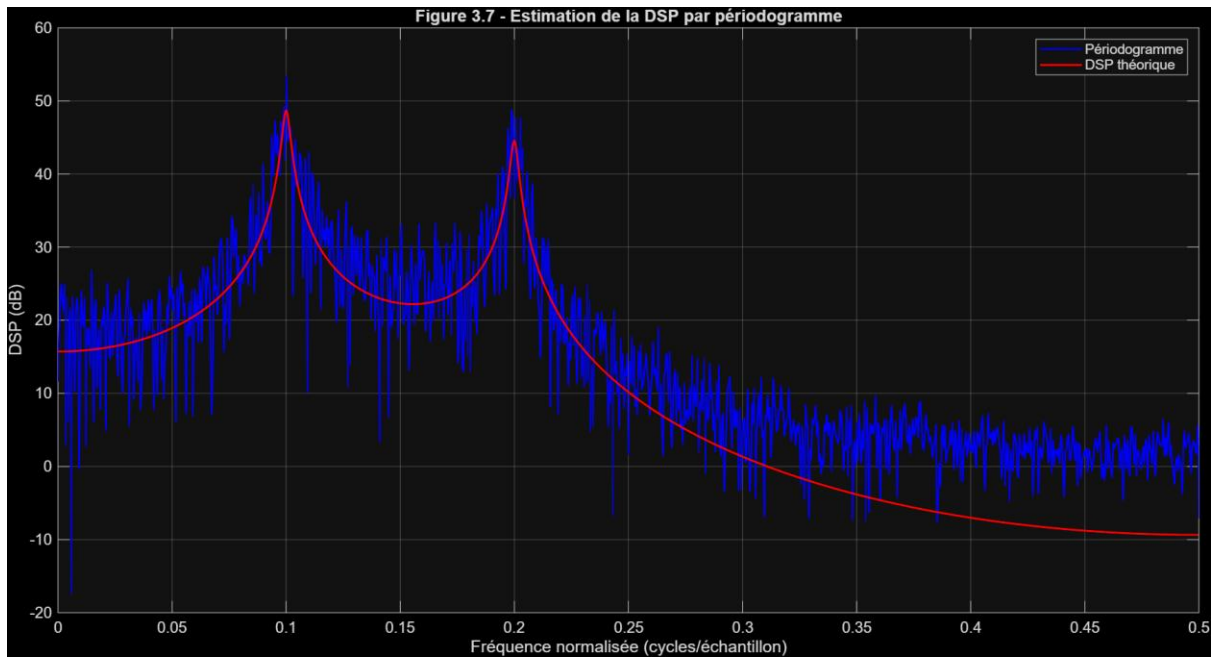


Cette courbe constitue la référence de comparaison pour les différentes méthodes d'estimation.

3.7.2 Estimation par périodogramme

```
[Sp, fp] = periodogram(yN, rectwin(length(yN)), 1024);

figure;
plot(fp/pi, 10*log10(Sp), 'b');
hold on;
plot(w/pi, 10*log10(S_theo), 'r', 'LineWidth', 1.5);
grid on;
legend('Périodogramme', 'DSP théorique');
title('Estimation de la DSP par périodogramme');
xlabel('Fréquence normalisée (\times \pi rad/échantillon)');
ylabel('DSP (dB)');
```

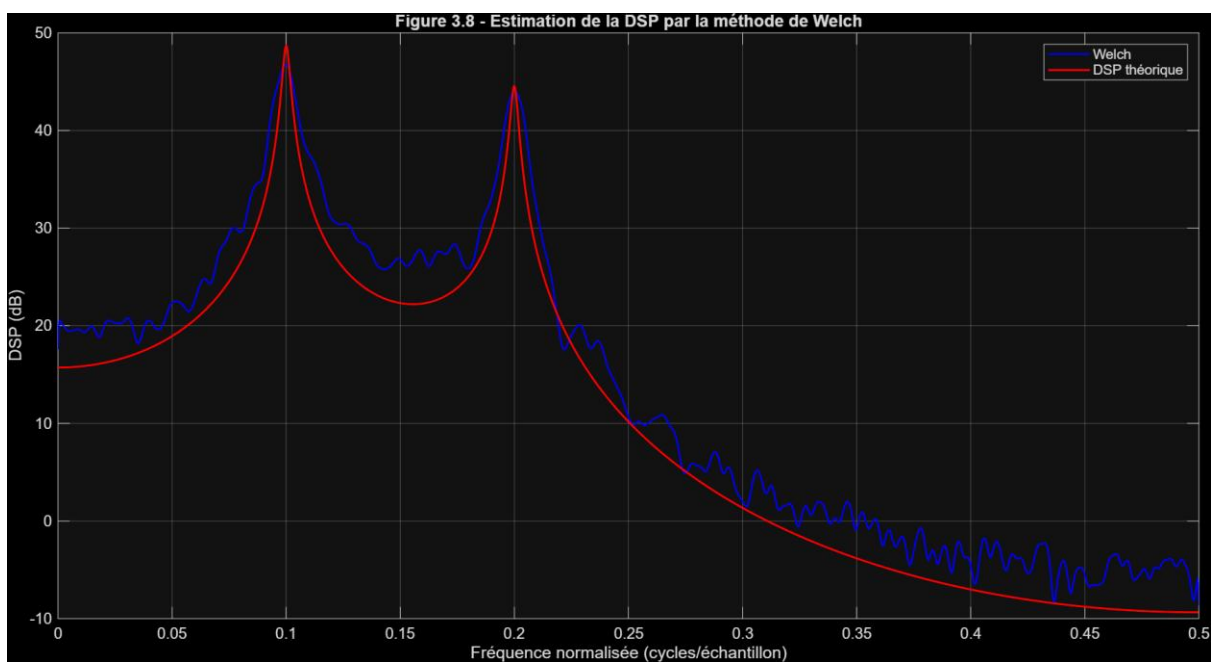


Le périodogramme suit globalement l'enveloppe de la DSP théorique, mais présente de fortes fluctuations. Il fournit donc une estimation simple, mais de variance élevée.

3.7.3 Estimation par la méthode de Welch

```
[Sw, fw] = pwelch(yN, [], [], 1024);

figure;
plot(fw/pi, 10*log10(Sw), 'b');
hold on;
plot(w/pi, 10*log10(S_theo), 'r', 'LineWidth', 1.5);
grid on;
legend('Welch', 'DSP théorique');
title('Estimation de la DSP par la méthode de Welch');
xlabel('Fréquence normalisée (\times \pi rad/échantillon)');
ylabel('DSP (dB)');
```



La méthode de Welch fournit une courbe nettement plus lisse que le périodogramme grâce au moyennage sur plusieurs segments. Elle réduit la variance de l'estimation, au prix d'une résolution fréquentielle un peu moins fine.

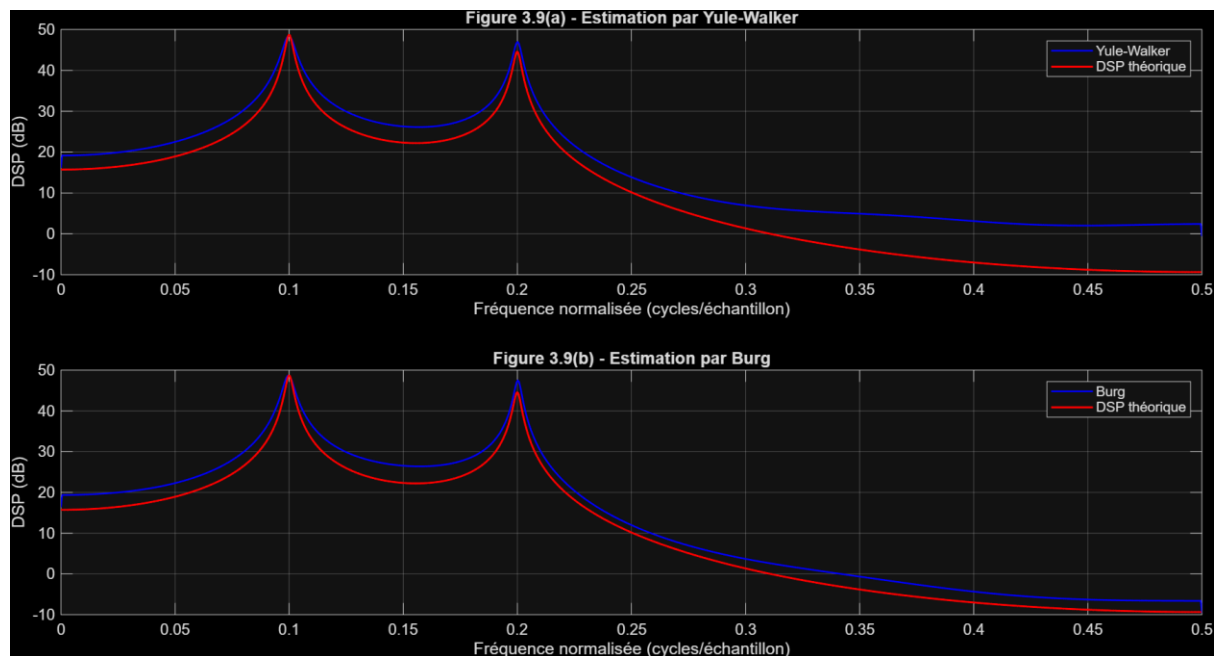
3.7.4 Estimation par les méthodes paramétriques

Les méthodes de Yule-Walker et de Burg ont été appliquées en supposant un modèle autorégressif d'ordre 10 :

```
[Syw, fyw] = pyulear(yN, 10, 1024);
[Sb, fb] = pburg(yN, 10, 1024);

figure;
subplot(2,1,1);
plot(fyw/pi, 10*log10(Syw), 'b');
hold on;
plot(w/pi, 10*log10(S_theo), 'r', 'LineWidth', 1.5);
grid on;
legend('Yule-Walker', 'DSP théorique');
title('Estimation de la DSP par Yule-Walker');
xlabel('Fréquence normalisée (\times \pi rad/échantillon)');
ylabel('DSP (dB)');

subplot(2,1,2);
plot(fb/pi, 10*log10(Sb), 'b');
hold on;
plot(w/pi, 10*log10(S_theo), 'r', 'LineWidth', 1.5);
grid on;
legend('Burg', 'DSP théorique');
title('Estimation de la DSP par Burg');
xlabel('Fréquence normalisée (\times \pi rad/échantillon)');
ylabel('DSP (dB)');
```



Les méthodes paramétriques donnent des estimations plus régulières que le périodogramme. Elles peuvent mieux suivre certaines structures spectrales, en particulier lorsque le signal est bien représenté par un modèle AR. En revanche, leur qualité dépend directement du choix de l'ordre du modèle.

3.7.5 Comparaison des méthodes

La comparaison des différentes méthodes met en évidence les points suivants :

- le périodogramme est simple à mettre en œuvre, mais présente une forte variance ;
- la méthode de Welch améliore nettement la stabilité de l'estimation ;
- les méthodes de Yule-Walker et de Burg fournissent des spectres plus lisses ;
- les méthodes paramétriques sont performantes lorsque le modèle choisi est adapté au signal ;
- la comparaison à la DSP théorique permet d'évaluer la fidélité réelle de chaque méthode.

Globalement, la méthode de Welch constitue un bon compromis entre simplicité et robustesse, tandis que les méthodes paramétriques peuvent être plus performantes sur certains signaux mais restent sensibles au choix du modèle.

3.8 Conclusion du chapitre

Ce troisième travail pratique nous a permis d'étudier les propriétés statistiques et spectrales des signaux aléatoires. L'analyse du bruit blanc gaussien a montré que ses estimateurs de moyenne et de variance convergent vers leurs valeurs théoriques lorsque la longueur d'observation augmente. L'étude de l'autocorrélation a également confirmé qu'un bruit blanc présente une corrélation concentrée autour du retard nul.

L'analyse des signaux MA et AR a mis en évidence l'effet du filtrage numérique sur la structure temporelle des signaux. Dans le cas MA, les corrélations restent localisées autour de quelques retards, tandis que dans le cas AR, la fonction d'autocorrélation décroît progressivement, traduisant la mémoire introduite par le filtre récursif.

Enfin, l'estimation de la densité spectrale de puissance a montré que les différentes méthodes étudiées ne présentent pas les mêmes performances. Le périodogramme est très fluctuant, la méthode de Welch est plus stable, et les méthodes paramétriques offrent des estimations plus lisses lorsque le modèle adopté est pertinent. La comparaison avec la DSP théorique a permis de valider ces observations et de mieux comprendre les avantages et les limites de chaque approche.

Ainsi, ce chapitre complète l'étude du traitement numérique du signal en introduisant les outils essentiels à l'analyse des signaux aléatoires et à l'estimation spectrale.

Conclusion générale

Au terme de ces travaux pratiques, nous avons pu consolider les notions essentielles du traitement numérique du signal à travers une approche à la fois théorique et expérimentale. Les différentes manipulations réalisées sous MATLAB ont permis de mieux comprendre les principes de représentation, de transformation, de filtrage et d'analyse statistique des signaux.

Dans le premier TP, l'étude de la quantification, de la convolution et de la transformée de Fourier discrète a montré l'importance des outils de base utilisés en traitement numérique. Nous avons notamment mis en évidence l'effet du nombre de bits sur la qualité de représentation d'un signal, la validité de la convolution temporelle et fréquentielle, ainsi que l'intérêt de la TFD pour identifier les composantes spectrales d'un signal.

Le deuxième TP nous a permis d'analyser plusieurs structures de filtres numériques et de comparer leurs performances. Cette partie a montré que le choix d'un filtre dépend fortement du compromis recherché entre sélectivité, régularité de la réponse, stabilité et phase. Elle a également mis en évidence l'intérêt des filtres RIF pour les applications nécessitant une phase linéaire.

Enfin, le troisième TP a permis d'introduire l'étude des signaux aléatoires et de relier les propriétés statistiques d'un signal à son comportement spectral. L'analyse des signaux MA et AR ainsi que la comparaison des méthodes d'estimation de la densité spectrale de puissance ont montré que la qualité d'une estimation dépend à la fois du modèle adopté, de la longueur d'observation et de la méthode utilisée.

En conclusion, ces travaux pratiques ont constitué une application concrète et progressive des principaux concepts du traitement numérique du signal. Ils ont permis non seulement de valider les résultats théoriques vus en cours, mais aussi de développer une meilleure maîtrise des outils de simulation et d'interprétation indispensables à l'analyse des signaux numériques.